UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

SOFTWARE ENGINEERING

Software Testing

THE WATERFALL SOFTWARE PROCESS MODEL

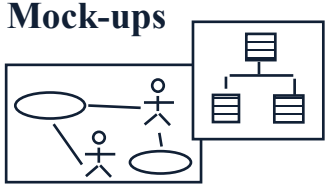
Requirements Engineering → **System Design** → **Software and UI Design** → **Implemen-tation** → **Testing** → **Operation and Maintenance**

Requirements collected via:

- Interviews with Stakeholders
- Personas
- Stories and Scenarios

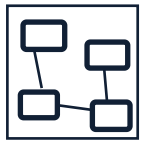
Specified using:

- Use Cases
- Natural Language
- Domain Models
- Mock-ups



Define System Architecture

- Requirements are allocated to software sub-systems
- Sub-systems are allocated to hardware resources
- Architectural Patterns

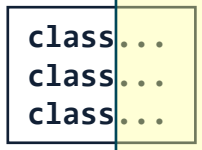


Define Subsystems

- Objects required to realize each subsystem are defined.
- Software Design Patterns
- Usability Engineering
- High-fidelity Wireframing

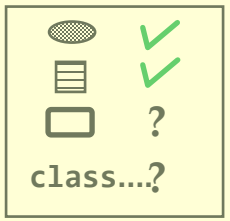
Each Subsystem is implemented

- Source code and other artifacts
- Clean Code
- Frameworks and ORMs
- Focus on Software Quality



Ensure the Software satisfies customers

- Code inspections
- Functional Testing (unit, integration, system testing)
- Usability Testing



System is put into practical use

Maintance will be required at some point

- To fix errors that were not discovered in previous phases
- To adapt the software to changes in requirements on in its environment

IEEE definitions for testing

IEEE 29119 - 2013 (reviewed and confirmed in 2019)

Testing: set of activities conducted to facilitate discovery and/or evaluation of properties of one or more test items.

Testing activities include planning, preparation, execution, reporting, and management activities, insofar as they are directed towards testing.

test item: work product that is an object of testing

EXAMPLES: A system, a software item, a requirements document, a design specification, a user guide.

Verify and Validation (V&V)

- Verify and Validation (V & V) show is the software satisfies its requirements and (Verification) and if it satisifies user requirements (Validation).
  - *Verification: Are we making the product right?*
  - *Validation: Are we making the right product?*
- Two complementary approaches to the verification:
  - Experimental approach (by testing or dynamical analysis of the software during its execution)
  - Analytic approach (static analysis, of code and documentation, formal analysis)

Definitions

- Verification: The process of determining whether the products of a phase of the software development process fulfill the requirements established during the previous phase.
- Validation: The process of evaluating software at the end of software development to ensure compliance with intended usage.

Classic Definitions

- **Error** (human error)
  - Human error in programming, due to wrong interpretations, insufficient knowledge of the problem to solve, material mistake or any other issue
- **Defect** (fault, bug)
  - Defect into the software code due to an error, that could cause the failure of the software system
- **Failure**
  - An execution of the software that does not provide the expected results and behaviour
  - Failures may be dynamic: they happens in a given time due to a given input and can be observed only via software execution

An example

```
int double(int x)
{
    y = x*x;

    return y;
}
```

ERROR
due to lack of knowledge or editing

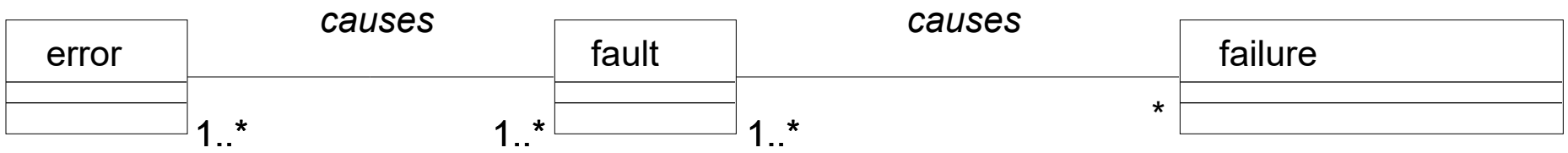
ERROR
We think that the double of x can be computed as x*x OR we have typed * instead of +

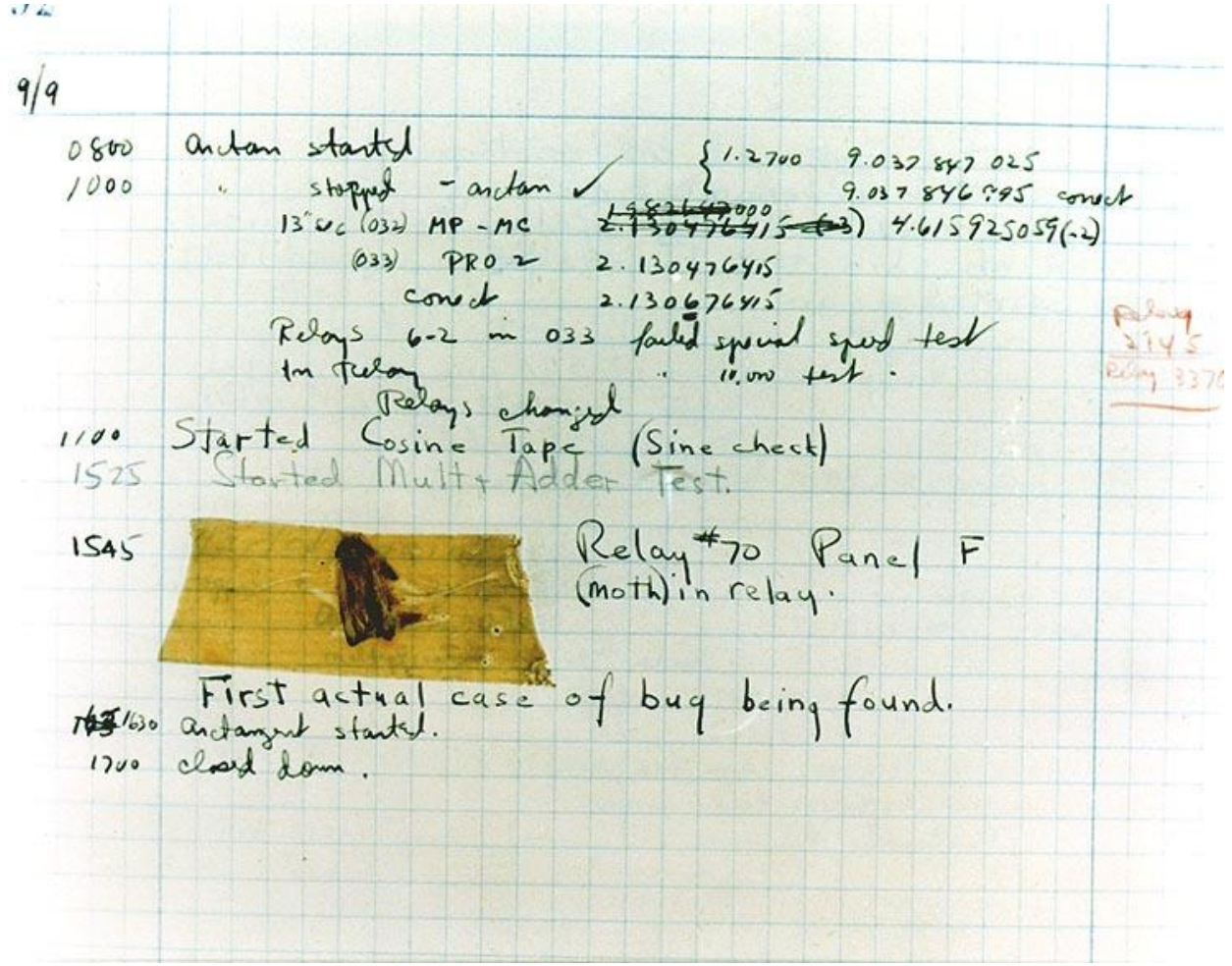
FAULT
"*" instead of "+"

FAILURE
The wrong result is shown

Test case 1 : 0 → 0 = 0 No failure
Test case 2 : 2 → 4 = 4 No failure
Test case 3 : 10 → 100 ≠ 20 Failure

Relationships between error, fault and failures





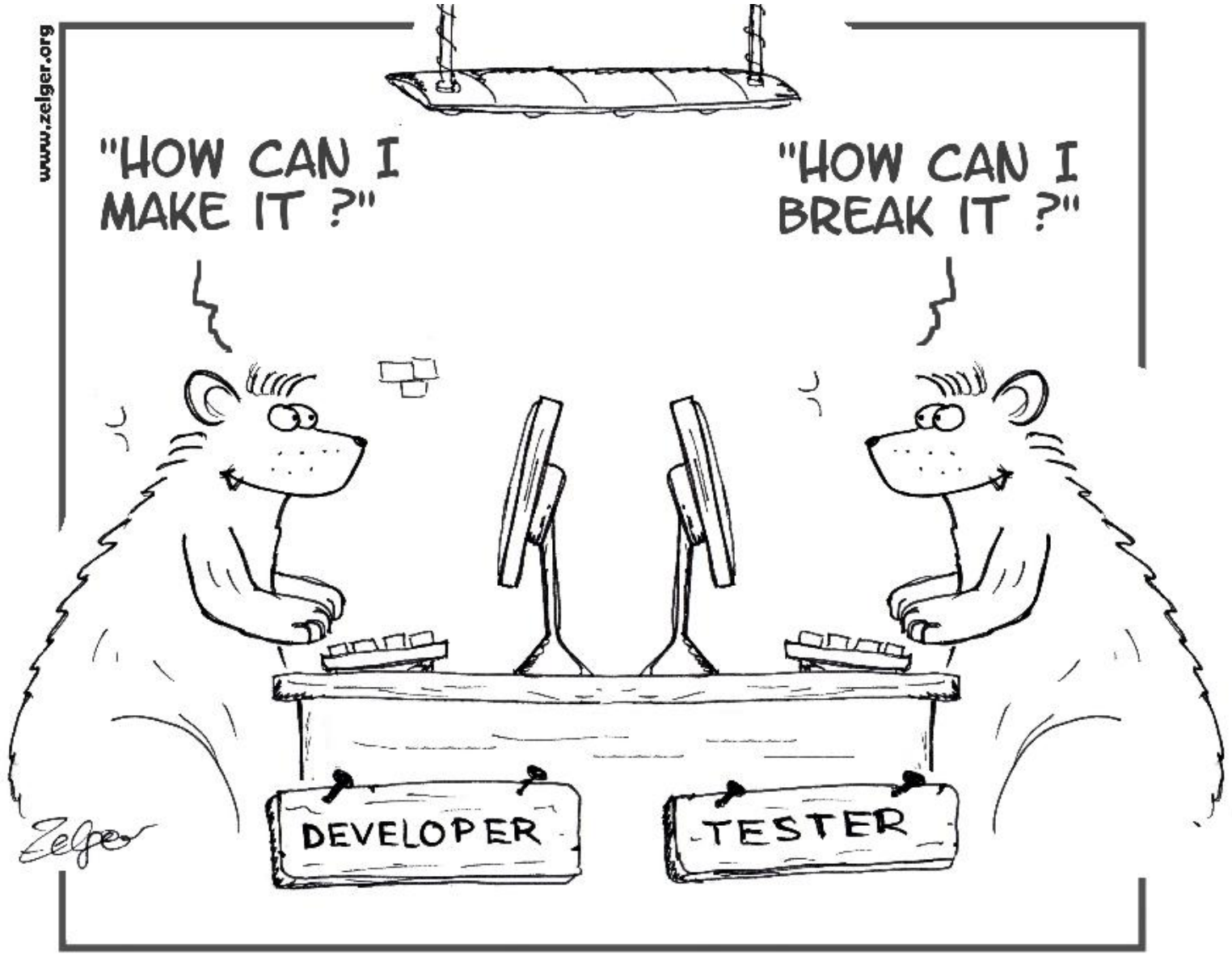
Grace Hopper's bug

- Note: probably the first use of the 'bug' term as a fault is due to Thomas Edison

Defect testing

- It is used to find faults in a software
  - Faults are found by searching for failures
- Defect testing *succeeded* if the program fails (i.e. failures are observed)
  - Why testing *fails* if no defects are found? It is a philosophical question

- Testing may demonstrate the presence of faults
  - May testing demonstrate the *absence* of faults, too?



They weren't so much different, but they had different goals

Dijkstra Thesis

- **Dijkstra Thesis**
  - Testing cannot prove absence of defects, but only their presence
- There is no guarantee that if at the n-th test a module or a system has responded correctly (i.e. no defects have been found), the same can be done at the (n+1)-th test
- Inability to produce all possible configurations of input values (test cases) at all possible internal states of a software system

The Seven Principles of Testing (from ISTQB)

- Principle 1 – Testing shows the presence of defects, not their absence
- Principle 2 – Exhaustive testing is (generally) impossible
- Principle 3 – Early testing saves time and money
  - The sooner the defect is discovered, the fewer changes need to be made to correct it
- Principle 4 – Defects tend to form clusters
  - A sort of adaptation of the principle of locality of the accesses
- Principle 5 – Beware of the pesticide paradox
  - The code "tends to adapt" to testing: in other words, to find new defects introduced over time it is always necessary to design new tests
- Principle 6 – Testing is context-dependent
  - There are no universal testing strategies and techniques applicable in every context
- -Principle 7 – The absence of errors is a false belief
  - Practical consequence of principles 1 and 2

Testing

Test Specification, Quality and Taxonomy

Test case specification

- Minimal set of information able to describe a test case specfication
  - ID Number and Description
  - Preconditions
    - *Hypotheses that must be veriified before the test is executed*
  - Input values
  - Expected output values (the «Oracle»)
  - Expected Postconditions
    - *Hypotheses that must be verified after the test is executed*

ID	Precond	Input	Expected Output	PostCond

Test case execution report

- In addition to the information needed for test case specification:
  - Output values
  - Result
    - *Success if preconditions and postconditions are true and output values are judged equivalent to expected output values*
    - *Failure if preconditions and postconditions are true but output values are not judged equivalent to expected output values*
    - *Not applicable (N/A) if al least a precondition or a postcondition is not true*

ID	Precond	Input	Expected Output	Output	PostCond	Result

Input

- Input can be described as attribue-value pairs …
- … or by a stream
  - For example, they can be represented by a text file
- Input may be described by a sequence of actions
  - For example in GUI based applications, input may be represented by a sequence of user actions
- Input may also be described by a set of signals and by the time of arrival to the system / exit from the system

Input

- Some examples:
  - Testing of methods
    - The list of variable is known, the type of values is known
  - Testing of (Web) services
    - The list of variable is known but the type of values is generic (string)
  - Testing of GUI based programs
    - Input is composed of a list of events and input values, the length of the sequence may assume any possible value
  - …

Example: method test case specification

- Method prototype:
  - int sum (int x, int y)

- Example of Test case Input Specifications

ID	Precond	Input	Expected Output	PostCond
		x=1 y=2		

- Input is specified as a set of pairs (name, value)

Example: method test case specification

- Method prototype:

  void increase (Integer x)

- Example of Test case Input Specifications

ID	Precond	Input	ExpectedOutput	PostCond
		x=new Integer(1)		

- Input is a specified as an object with an indication of the method (constructor) needed to obtain it
  - The input value is a parameter of this call

Example: GUI based program test case specification

- Program to test
  - notepad.exe
  - Function to test: search the word «pippo» into the text

- Example of Test case Input Specifications

ID	Precond	Input	ExpectedOutput	PostCond
		Open notepad Click on Modify Click on Find.. Type «pippo» Click on Find Next Exit Notepad		

- Input is specified as a sequence of events to be triggered on the GUI

Output

- The output may be specified:
  - As values/objects in method / services testing
  - As a graphical screen / object on the screen in GUI program testing
  - …

Example: static method test case specification

- Method prototype:
  - static int sum (int x, int y)

- Example of Test case Output Specifications

ID	Precond	Input	Expected Output	PostCond
		x=1 y=2	3	

- Output is specified as a int value

Example: method test case specification

- Method prototype:

  class Count

  void increase (Integer x)

- Example of Test case Output Specifications

ID	Precond	Input	ExpectedOutput	PostCond
		x=new Integer(1)	None	

- There is no actual output
  - The method execution causes a change in the internal state of the object of the class Count but not in the method output (that is void)

Example: GUI based program test case specification

- Program to test
  - notepad.exe
  - Function to test: search the word «pippo» into the text of the file prova.txt

- Example of Test case Output Specifications

ID	Precond	Input	ExpectedOutput	PostCond
		Open notepad prova.txt Click on Modify Click on Find.. Type «pippo» Click on Find Next Exit Notepad	"pippo found" text message	

- Output is specified as a widget shown on the GUI

Preconditions and postconditions

- Preconditions and postconditions may be about:
  - The existence and state of external services or data sources
    - *For example files, databases, services, global variables …*
  - The state of the application before/after the test execution
    - *For example: the correct authentication, the presence or absence of such data in the database, the reaching of a specific interface*
      - ***Usually, if a test is quite complex, then intermediate conditions are added to improve the fault localization ability of the test case***

Example: method test case specification

- Method prototype:
  - static int sum (int x, int y)

- Example of Test case Precondition Specification

ID	Precond	Input	Expected Output	PostCond
	None	x=1 y=2	3	

- The method has no state : no precondition are needed

Example: method test case specification

- Method prototype:

  class Count

  void increase (Integer x)

- Example of Test case Precondition Specifications

ID	Precond	Input	ExpectedOutput	PostCond
	Count c exists c.value==2	x=new Integer(1)	None	

  - Since triple acts on a non-static object of the class Count, the precondition regards its existence:
    - An object c of the class Count has been instantiated

Example: GUI based program test case specification

- Program to test
  - notepad.exe
  - Function to test: search the word «pippo» into the text of the file prova.txt

- Example of Test case Precondition Specification

ID	Precond	Input	ExpectedOutput	PostCond
	prova.txt exists prova.txt contains the word 'pippo'	Open notepad prova.txt Click on Modify Click on Find.. Type «pippo» Click on Find Next Exit Notepad	"pippo found" text message	

- In this case there are two different preconditions: the logic AND of both the two have to be verified to allow the test case execution

Example: method test case specification

- Method prototype:
  - static int sum (int x, int y)

- Example of Test case PostCondition Specification

ID	Precond	Input	Expected Output	PostCond
	None	x=1 y=2	3	None

- The method has no state : no postconditions are needed

Example: method test case specification

- Method prototype:

  class Count

  void increase (Integer x)

- Example of Test case Precondition Specifications

ID	Precond	Input	ExpectedOutput	PostCond
	Count c exists c.value==2	x=new Integer(1)	None	c.value==3

- The method increase adds x=1 to the current value of object c
  - If the precondition is that c.value ==2 then the postcondition is that c.value==3

Example: GUI based program test case specification

- Program to test
  - notepad.exe
  - Function to test: search the word «pippo» into the text of the file prova.txt

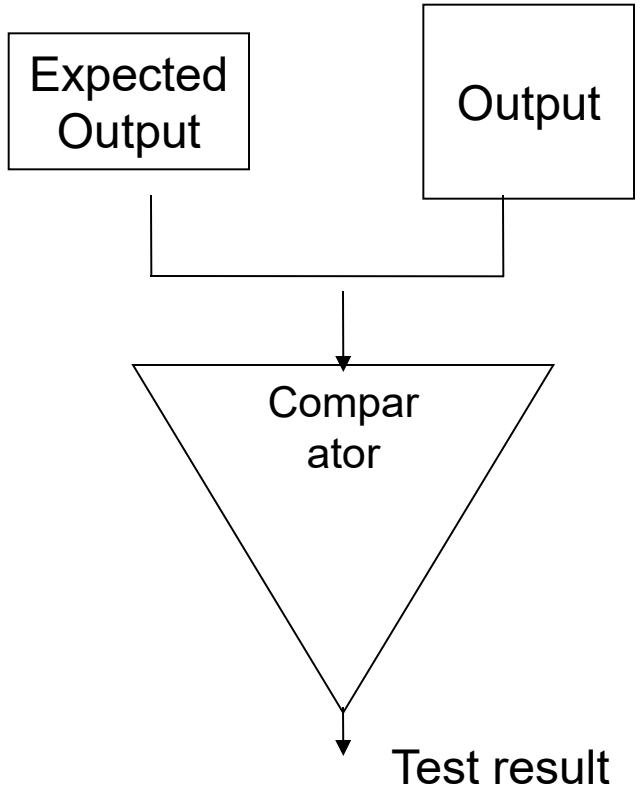
- Example of Test case Precondition Specification

ID	Precond	Input	ExpectedOutput	PostCond
	prova.txt exists prova.txt contains the word 'pippo'	Open notepad prova.txt Click on Modify Click on Find.. Type «pippo» Click on Find Next Exit Notepad	"pippo found" text message	"pippo found" text message is shown on the screen

- In this case the find operation does not modify the file, thus no postconditions regards the file
- It is necessary to specify the state of the GUI to allow the execution of a different test case after this one
  - For example, a text substitution test case

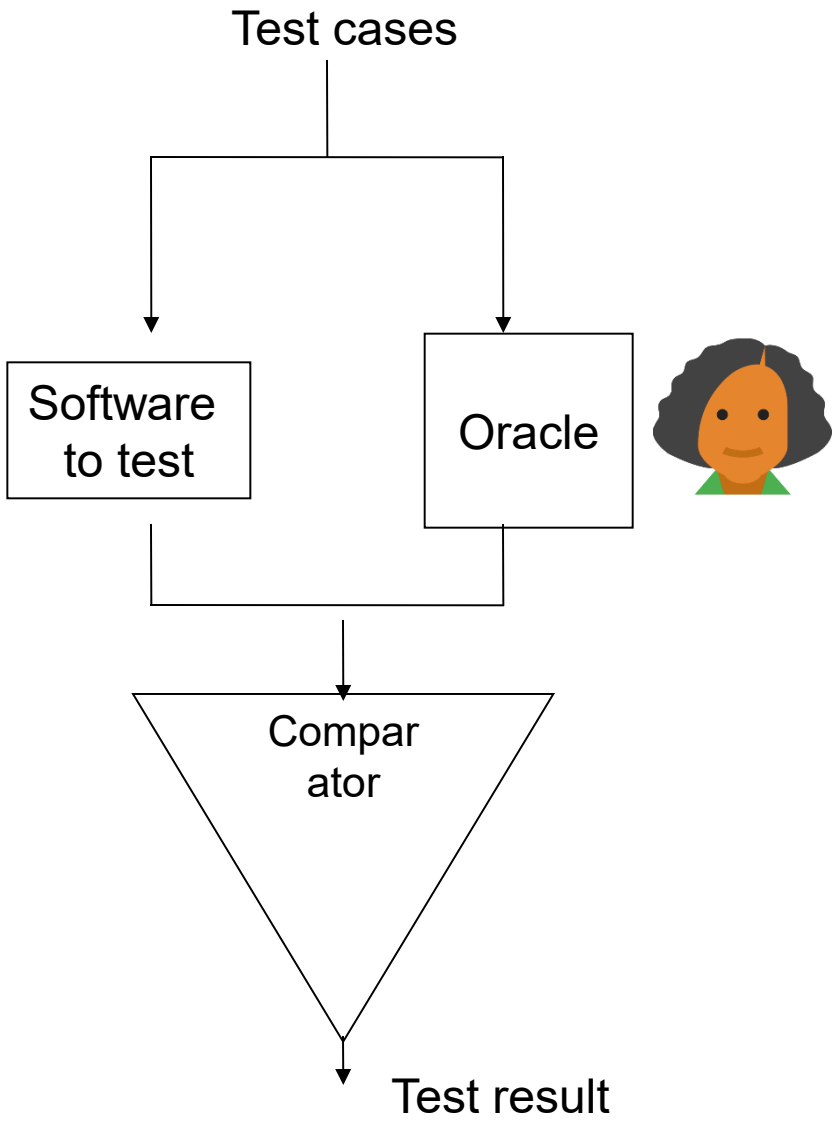
Output, Expected Output, Test Result

- The (Actual) **Output** obtained by the test execution has to be compared with the **Expected Output** designed in the test case in order to evaluate the test Result
- The **Comparator** is able to compare the oracle expected behavior and the tested software behavior and to evaluate if a failure has occurred
- The Comparator has to evaluate *objectively* if the Expected Output and the Output are equal / identical / equivalent between them



the Oracle

- the **Oracle** knows exactly what are the Expected Outputs of the system under test in response to the each test case
- In the easiest scenario, the Oracle is represented by the column of the Expected Outputs



Who is the Oracle?

- Human oracle
  - He evaluates the success of the test cases on the basis of the requirements and of his personal judgement
    - *For example, in acceptance testing*
- Software oracle
  - An Oracle is another software having the same behavior of the software to be tested
    - *For example, a known version of bubblesort can be used to test a faster quicksort*
    - *A previous version of a software can be the oracle to evaluate the behavior of a newer software offering the same functionality (regression testing)*
- Implicit oracle
  - Testing against crashes, the oracle found a failure in each case a crash occurs
- Formal specification oracle
  - If the requirements are expressed in a formal way, then the oracle can be automatically derived from them

Test Quality

Testing quality requirements

1. Accuracy
   - The evaluation of the result should be accurate and objective
2. Repeatability
   - Test cases should be repeated in the same conditions and must give the same result each time
3. Robustness
   - Test cases could be reused even after program modifications
4. Fault Localization Ability
   - Test failures guides the debugging process
5. Traceability
   - Test cases should be linked to requirements and design
6. Effectiveness
   - Test cases should find all the faults
7. Efficiency
   - The test suite should find the faults with a minimal number of test cases

Testing Quality Requirements: effectiveness

- The (main) objective of the testing activities is to find the largest number of faults
  - A test suite T1 is more effective than another test suite T2 if it is able to find more different faults
    - *But it is not implied that if T1 has more test cases founding faults, it will find more different faults*
    - *And it is not implied that if T1 finds more failures, it will find more (different) faults*
    - *An obvious strategy to increase effectiveness is to execute more and more test cases …*

Effectiveness measures

- **Effectiveness** can be measured as:
  Number of different faults found / Number of existing faults

  - … but the number of existing faults is generally unknown
  - … and the number of executed test cases is not a measure of the effort related to their execution
  - … and test cases founding all the same fault should not contribute positively to efficiency

  Relative Effectiveness (T1, T2) = Number of different faults found by T1 / Number of different faults found by T2

Approximate measures: code coverage

- The «exact» measures of effectiveness and efficiency depend on the real presence of faults. If a software has no residual bugs, the effectivess and efficiency of each test suite are always zero
  - But we do not know if there are any bugs, so we cannot avoid testing
  - We need approximate measures of effectiveness and efficiency useful in cases there few or no bugs

- An approximate measure of effectiveness is code coverage:
  - CovLOC=Number of LOCs covered by at least a test case / Total number of LOCs
  - CovMethod=Number of methods called by at least a test case / Total number of methods
  - CovClass= Number of classes used by at least a test case /Total number of classes

Code coverage ratio

- If a LOC contains a fault and no tests execute it, then the fault SURELY will not be found
- If a LOC contains a fault and some tests execute it, then it is POSSIBLE that the fault will cause a failure

- In general, more the quantity of code that we execute, more the «hope» to found faults
- If two test suites T1 and T2 will found the same number of faults (possibly 0 faults), and T1 cover more LOCs than T2, we could have a better confidence in test suite T1

Testing Taxonomy

Testing taxonomy

- There are many different possible taxonomies of testing related activities
  - With respect to the applied testing technique
  - With respect to the objects under test
  - With respect to the tested properties
  - With respect to the testing processes
  - …
- There are also different standards proposing different testing taxonomies

IEEE 29119 Taxonomy

- Four dimensions:
  - Verification and validation techniques
  - Levels of Testing
  - Testing processes
  - Tested properties

Verification and Validation Techniques

Verification and Validation	Static Testing	Reviews		
		Model Verification		
		Static analysis		
	Dynamic Testing Test Design Techniques	Specification-based	Equivalence Partitioning	
			Classiication Tree Method	
			Boundary Value Analysis	
			Syntax Testing	
			Combinatorial Test Design Techniques	All Combinations Testing
				Pair-Wise Testing
				Each Choice Testing
				Base Choice Testing
			Decision Table Testing	
			Cause-Effect Graphing	
			State Transition Testing	
			Scenario Testing	
			Random Testing	
		Structure-based	Statement Testing	
			Branch Testing	
			Decision Testing	
			Branch Condition Testing	
			Branch Condition Combination Testing	
			Modified Condition Decision Coverage Testing	
			Data Flow Testing	All-Deinitions Testing
				All-C-Uses Testing
				All-P-Uses Testing
				All-Uses Testing
				All-DU-Paths Testing
		Experience-based	Error Guessing	
	Demonstration	Prototyping		
		User trials		
		Mock-ups		
	V and V analysis	Formal Methods	Model Checking	
			Proof of correctenss	
		Simulation		
		Evaluation	Quality metrics	

Testing levels

Levels of Testing	Unit Testing	
	Integration Testing	
	System Testing	
	System Integration Testing	
	Acceptance Testing	User acceptance testing
		Operational acceptance testing
		Factory acceptance testing
		Alpha testing
		Beta testing
		Production verification testing

Unit testing

- Unit by definition is the smallest part of the software you intend to test
  - The concept of Unit coincides with the concept of module used in the design
- Units can be:
  - Functions
  - Methods
  - Classes
  - Packages (note their interfaces)
  - The entire system
- Unit testing can be carried out:
  - In black box mode, if only the specification of the unit is known (inputs/outputs/function realized)
  - In white box mode if you also know how the unit is made and you want to exploit this knowledge to evaluate even the partial correctness of the unit

Integration Testing

- A unit can only be tested if we can isolate it from the rest of the system
  - Isolation testing is achieved by implementing components that replace the components or services on which the unit under test depends with test doubles (e.g. mocks or stubs)
- Integration testing allows you to test a system bit by bit, possibly using test doubles

System Testing

- System testing is testing a software system in its entirety
  - For example, by interacting with the user interface and observing results both on the interface and, for example, on databases or files

Testing practices / processes

Test practices	Model-based testing	
	Scripted testing	
	Exploratory testing	
	Experience-based testing	
	Manual testing	
	A/B testing	
	Back-to-back testing	
	Mathematical-based testing	
	Fuzz testing	
	Keyword-driven testing	
	Automated testing	Capture-replay driven
		Data-driven

Testing objectives

Types of testing	Functional testing	
	Accessibility testing	
	Compatibility testing	
	Disaster/recovery testing	
	Installability testing	
	Interoperability testing	
	Localization testing	
	Maintainability testing	
	Performance-related testing	Performance
		Load
		Stress
		Capacity
		Recovery
	Portability Testing	
	Procedure Testing	
	Reliability Testing	
	Security Testing	
	Usability Testing	

Software Verification and Validation

Black Box Testing

Specification-based testing

- **Principle of requirements verifiability:**
  - **requirements should be testable**
    - ***written so that tests can be designed that prove that the requirement has been met.***
- **Requirements-based testing is a validation technique where various tests are designed for each requirement.**

- **Generally speaking, specification-based testing is included in the large family of black box testing**
  - **Black box testing is intended as a testing technique in which test case are designed without any internal information about the system under test, i.e. based only on its specification**

Example: Use case scenarios testing

- **Given the Use Case Diagram and the description of all use case scenarios**
  - **For each scenario, you design one or more test cases that run it**
  - **You run designed test cases manually or automatically**
  - **Testing strategy aims to cover use cases and scenarios**

Requirement testing vs Input-domain partitioning

- Two complementary approaches :
  - In requirements-based testing, test cases are designed on the basis of the analysis of the behaviors that you want to verify
    - *The difficulty is to find test values that are able to trigger them*
  - In Input-domain partition techniques (i.e. data-driven testing), test cases are designed to cover combinations of input values
    - *The difficulty consists in being able to cover, possibly with the minimum number of test cases, the behaviors of the application*

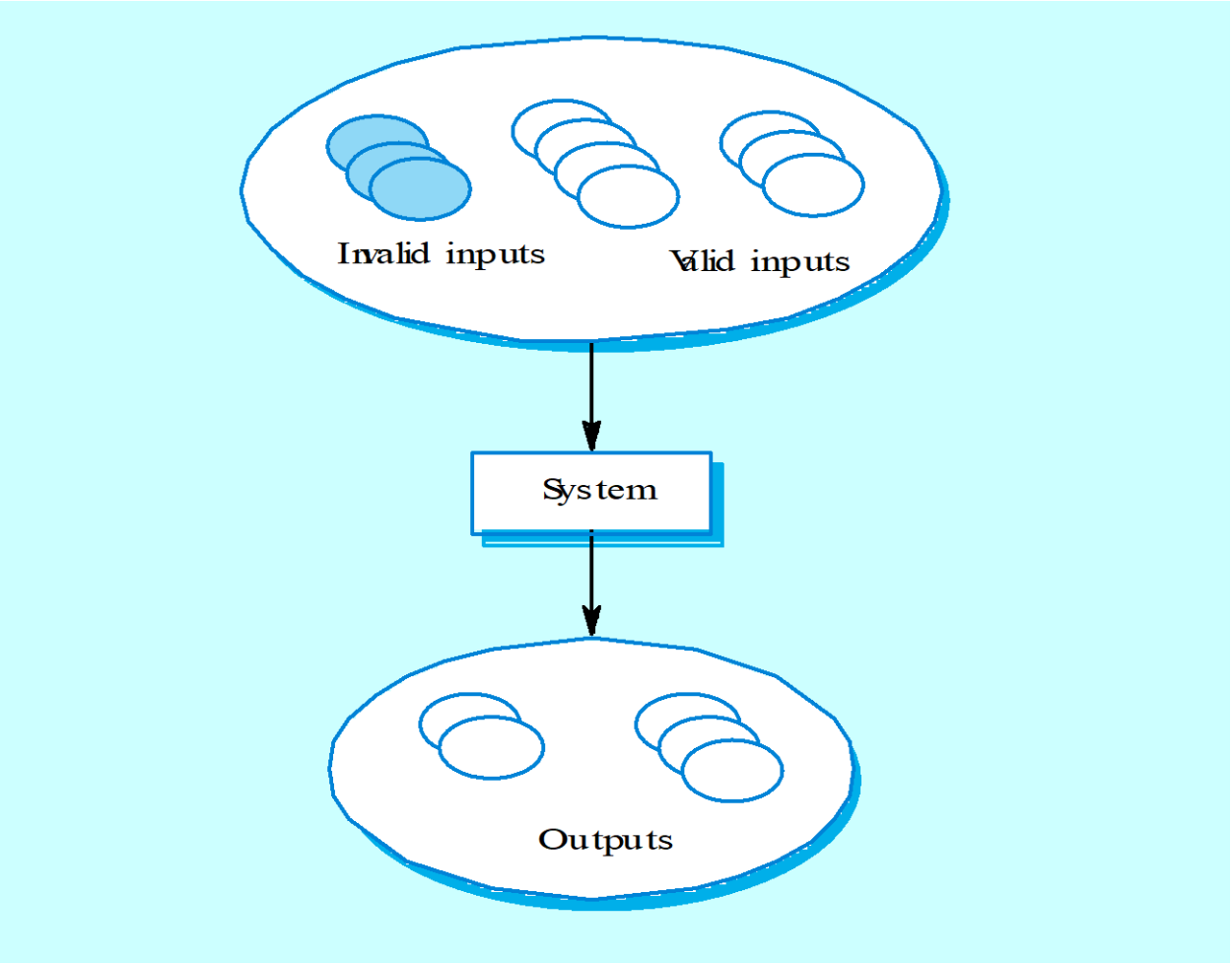
2- Input Domain Partitioning techniques

- **Input fields value can be divided into classes where all members of the same class are somehow related.**
  - **Each of the classes constitutes an equivalence class**
- **The rationale is that we hypothesize that the program should behave in the same way for each member of the class.**
- **Test cases consists of a selection of values belonging to equivalence classes**
  - **Coverage criteria can be designed based on the coverage of the equivalence classes for each input**

Equivance class partitioning

- **The equivalence classes for an input represent a partition of the input domain:**
  - **The union of all the equivalence classes corresponds to the input domain**
  - **There are no intersections between the equivalence classes**

Equivalence partitioning



Example

- In a method you have to enter a date, consisting of day (integer), month (string), year (integer)
- The method must correctly recognize between valid dates (corresponding to days that actually existed) and invalid dates
- The method returns a boolean (valid date if true) and a string (the day of the week or "Error" in case of invalid date)

Domain information

- **Source:** https://it.wikipedia.org/wiki/Calendario_gregoriano
- **In particular:**
  - **Months have different durations**
  - **February 29th exists only in leap years**
  - **Leap years are divisible by 4, but not by 100**
  - **The calendar came into force on October 15, 1582**
    - ***For didactic reasons, we introduce these constraints progressively in the examples***

More details

- Input:
  - Day – integer in a limited range below and above
  - Month – string in a set including 12 valid values
  - Year – integer in a bounded range below
- Output
  - Validity – boolean
  - Day of the week – string in a set including 7 valid values and an invalid value (Error)

Approaches for equivalence class definition

1. Interface-based approach
   - Input parameters are considered in isolation
     - *For each of them classes of inputs are designed*
2. Functionality-based approach
   - Identify characteristics that correspond to the intended functionality
   - Equivalence classes are thought with respect to the expected behavior
   - Takes into account relationships among parameters
   - Modeling is based on requirements, not on implementation!

1. Interface-based approach

- **Partitions are identified using program specifications or other documentation.**
- **The equivalence class represents a set of values for a given input variables, under the hypothesis that:**
  - **the behaviour of the system under test will be the same for different values belonging to the same equivalence class**

Searching for equivalence classes

- For each input you get
  - One or more valid equivalence classes, corresponding to the set of values considered valid for that input
  - A set of invalid equivalence classes, one for each invalid condition. Each of these conditions corresponds to a set of values (invalid equivalence class)

- We are considering only one input field at a time, ignoring the possible existence of interactions between the selection of values for different inputs

General cases

- If the input is a:
  - range of values
    - *a valid class for values within the range, an invalid class for values below the minimum, and an invalid class for values above the maximum*
  - specific value
    - *a valid class for the specified value, an invalid class for lower values, and an invalid class for higher values*
  - element of a discrete set
    - *a valid class corresponding to the collection (minimal testing) or a valid class for each element of the collection (detailed technique), an invalid class for an element not belonging to that set*
  - A boolean value
    - *As in the previous case, but for a discrete set of two values (true, false)*
- In some cases, an extra invalid class including all the remaining values that could be set as input should be considered

Valid and invalid classes: an example

Let's consider only a minimal set of domain information, related to the function interface:

- Day – integer in a limited range below and above (1..31)
  - *{day≤0}, {1≤day≤31}, {day>31}*
- Month – string in a set of 12 values (January..December)
  - *{month ∈ {January, February, March, April, May, June, July, August, September, October, November, December}}, {month ∉ {January, February, March, April, May, June, July, August, September, October, November, December}}*
- Year – integer in a bounded range below (1582..)
  - *{year<1582}, {year>=1582}*

Valid and invalid classes

- Additional classes correspond to the values non-belonging to the type
  - for example, a string for input *day*
- These tests are necessary in case of testing systems without control over the type
  - For example, having an input stream, or web forms, or protocols
- but they are not required in the scope of unit testing
  - the compiler would prevent a function call with the wrong type

Warning

- Equivalence classes are sets of possible test values
- Test cases consists of the specification of one value for each input
  - The equivalence class in which the selected input value is, has been covered by the test case

Equivalence classes Coverage criteria

- Given the equivalence classes, we may fix testing criteria aiming at the coverage of them, in different ways:
  - Each Choice Coverage (ECC)
  - All Combinations Coverage (ACC)

Each Choice Coverage

- Minimum coverage of equivalence classes
  - Each equivalence class is covered by at least one test case
    - *Minimum number of test cases equal to the maximum number of input classes with multiple equivalence classes*
      - **Based on the interface-based example**
      - **We omit tc4 in case of compiled languages because the type check is done by the compiler so we cannot run tests with incorrect type values**

Test case	**TC1**	**TC2**	**TC3**	**TC4**
Day	1	0	35	first
Month	January	Brumaire	January	January
Year	1980	1492	1980	two thousand

All combinations coverage (ACC)

- Coverage of all equivalence class combinations
  - *The number of test cases equals to the product of the cardinalities of the quantities of equivalence classes of each class*
  - *Corresponds to the t-wise case when t=n =number of inputs*

Test case	TC1	TC2	TC3	TC4	TC5	TC6
Day	1	0	35	1	0	35
Month	January	January	January	Brumaire	Brumaire	Brumaire
Year	1980	1980	1980	1980	1980	1980
Test case	TC7	TC8	TC9	TC10	TC11	TC12
Day	1	0	35	1	0	35
Month	January	January	January	Brumaire	Brumaire	Brumaire
Year	1492	1492	1492	1492	1492	1492

2. Functionality-based approach

- We will consider more details about the specification of the calendar method
  - These details are more related to the knowledge of the expected behaviour of the function instead only to its interface
- For example, let's consider these constraint:
  - There are some months that may have only 28, 29, 30 or 31 days
- How can we improve the design of the equivalence classes for day and month ?

Valid and invalid classes: functionality-based approach

Input:

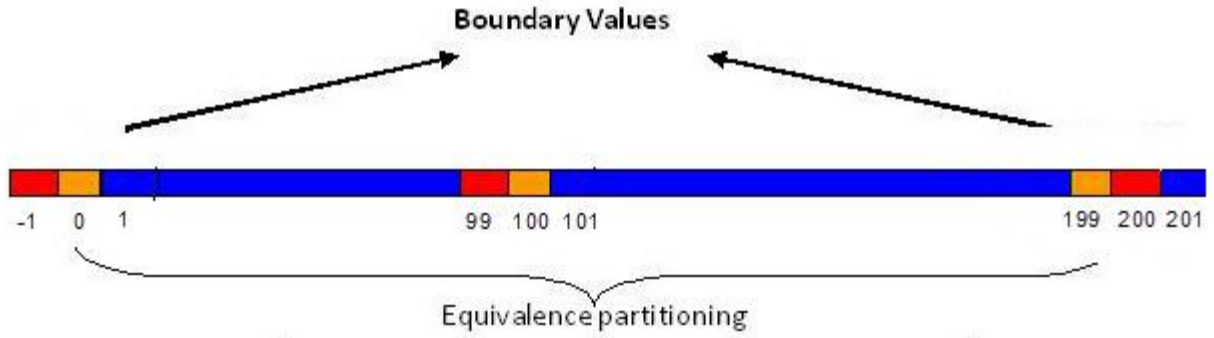
- Day
  - *{day≤0}, {1≤day≤28}, {day=29}, {day=30},{day=31}, {day>31}*
    - **6 classes**
- Month – string in a set of 12 values
  - *{month ∈ {January, March, May, July, August, October, December},*
  - *month ∈ {April, June, September, November},*
  - *month ∈ {February}},*
  - *{month ∉ {January, February, March, April, May, June, July, August, September, October, November, December}}*
    - **4 classes**
- Year – integer in a bounded range below
  - *{year<1582}, {year=1582}, {year>1582}*
    - **3 classes**

Boundary value analysis

- A variant to the equivalence class technique consider limit values (boundaries)
- Boundary values form additional valid and invalid equivalence classes corresponding to the limit values of the data validity sets
- Usually, boundary values regard intervals
- Boundary values are also values for which it is assumed that there may be a particular behavior with respect to some operation
  - for example the zero value for an integer that might fit into a division or for a pointer

Examples of boundary values

- If the condition on input variables specifies:
  - (Closed) range of values
    - *Boundary classes: minimum of the range, maximum of the range (valid classes), slightly below minimum, slightly above maximum (invalid classes)*
    - *If there are more ranges, the reasoning is iterated for each of them*
  - Integer values
    - *A boundary class, regardless of specification, is the set {0}; another, unless otherwise considered, is the class of negative numbers, and so on*



Examples of boundary classes

- For the input Day:
  1. **{Day <<0}: lower value of the lower bound of the range**
  2. {0}: slightly lower value of the lower bound of the range and also null value
  3. {1}: lower end
  4. {2}: value slightly higher than the lower bound
  5. **{Day >>2 && Day << 27}: valid value away from boundaries**
  6. {27}: value slightly lower than the upper bound
  7. {28}: upper bound in some cases
  8. {29}: well-known critical value
  9. {30}: well-known critical value
  10. {31}: well-known critical value
  11. {32}: value slightly greater than the upper bound
  12. **{Day >> 31}: value greater than the upper bound of the range**

- For the input input Year
  1. **{Day << 1582}: lower value of the lower bound of the range**
  2. *{1581}: slightly lower value of the lower bound of the range*
  3. *{1582}: lower bound and critical value*
  4. *{1583}: value slightly higher than the lower boun*
  5. ***{Year >> 1583}: value higher than the upper bound***

(non-boundary values in bold)

Examples of boundary classes

- For the input Month:
  1. **{Month ∉ months of the Year}**
  2. {January}
  3. {february}
  4. {march}
  5. {april}
  6. {may}
  7. {june}
  8. {july}
  9. {august}
  10. {september}
  11. {october}
  12. {november}
  13. {december}

(non-boundary values in bold)

The Oracle Problem

- Automatic generation of combinatorial test cases does not include the definition of oracles
  - They should be added by hand, increasing testing costs until it becomes impractical!

- The technique remains viable in some specific problems:
  - crash testing: the oracle (evaluated in a totally automatic way) consists in the evaluation of the regular termination of the program
    - *There would also be to consider the case of non-termination ...*
  - occurrence of invariant failures (e.g. security violations, privacy violations, excessive use of memory, violations of usability rules, etc.)

Structural testing (white box)

Structural-based testing

- White Box testing is a structural test, as it uses the knowledge of the internal structure of the program to derive test data.
- Observing code-level execution also provides objective criteria for evaluating the effectiveness of test cases
  - Code coverage metrics
- Very useful information for the localization of defects can be obtained by observing code execution

Comparison

- Functional (Black Box) testing is driven by the goal of covering as many behaviors of the application as possible, based on the knowledge we have of the ways in which we can interact with it (for example by varying input values)
- Structural (White Box) testing, on the other hand, is driven by the goal of running as much of the application code as possible, in order to be able to test all its code

Graph Modeling

- Many components of a software system may be modelled with graphs
  - Algorithms (flow diagrams)
  - Methods / functions (control flow graphs)
  - Interactions between classes (object diagrams)
  - Activity diagrams
  - Finite state machines
  - Statechart diagrams
  - …
- Let’s focus our attention on modelling the source code with control flow graphs

Control-Flow Graph

- Control-Flow Graph) of a program (method/function) P:
  - CFG (P) = <N, AC, nI, nF>

where:

<N, AC> is a direct graph with labeled edges,

$\{nI, nF\} \subseteq N$, $N - \{nI, nF\} = Ns \cup Np$

Ns and Np are disjunct sets of *statement nodes* and *predicate nodes (decisions)*;

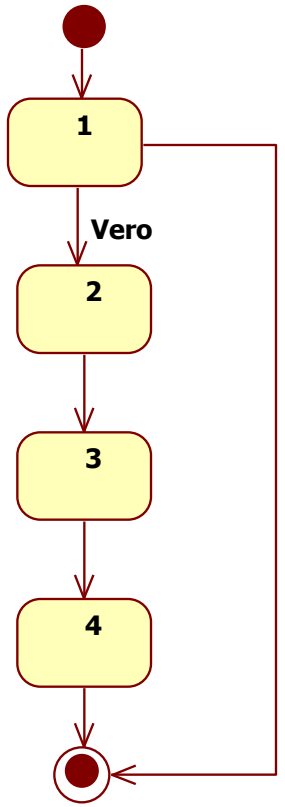
$AC \subseteq N-\{nF\} \times N-\{nI\} \times \{true, false, incond\}$ if the control flow relationship

nI are nF the *initial node* and the *final node*.

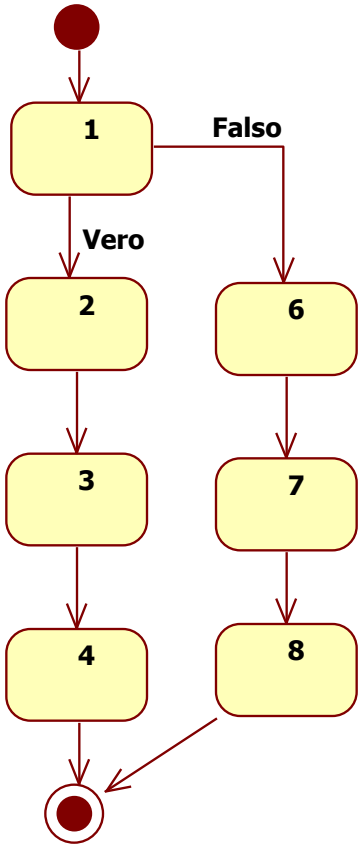
- A node $n \in Ns \cup \{nI\}$ has only a successor and its exiting edge is labeled as incond.
- A node $n \in Np$ has two successors ant its exiting edges are respectively labeled true and false.

Control structures on the CFG : if , if-else

```
1.   if
     (decisione)
2.   {
3.     Blocco
4.   }
```




```
1.   if
     (decisione
     )
2.   {
3.     Blocco1
4.   }
5.   else
6.   {
7.     Blocco2
8.   }
```



Observations

- CFG can be modelled from two different points of view:
  - Source code level
  - Executable (or bytecode) level

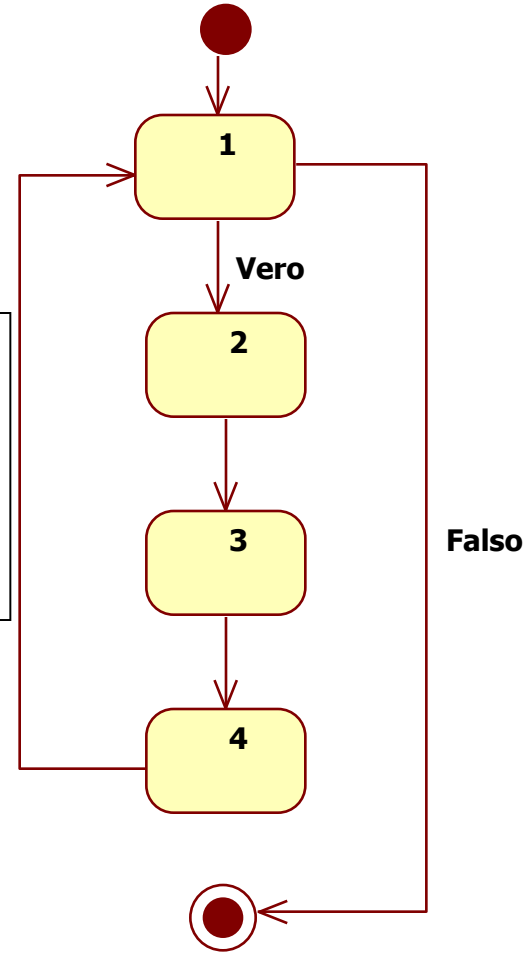
The nodes 2, 4, 6, 8 may be omitted thinking at the executable level since they do not correspond to any executable instructions

Source code level vs executable level

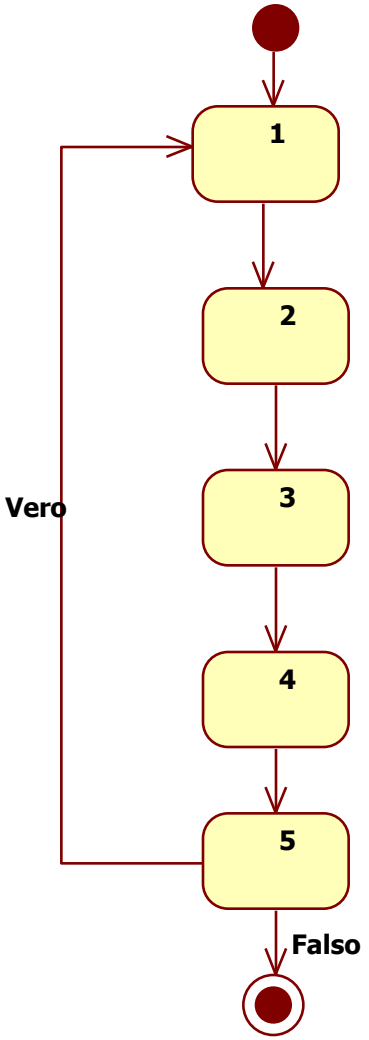
- Is it more useful to model at source code level or at executable code level?
  - Source code:
    - *it is easier to design tests that run specific parts of the source code*
    - *the graph is more compact and more readable*
  - Executable code:
    - *the graph represents exactly what is being performed*

Control structures on the CFG : while, do-while

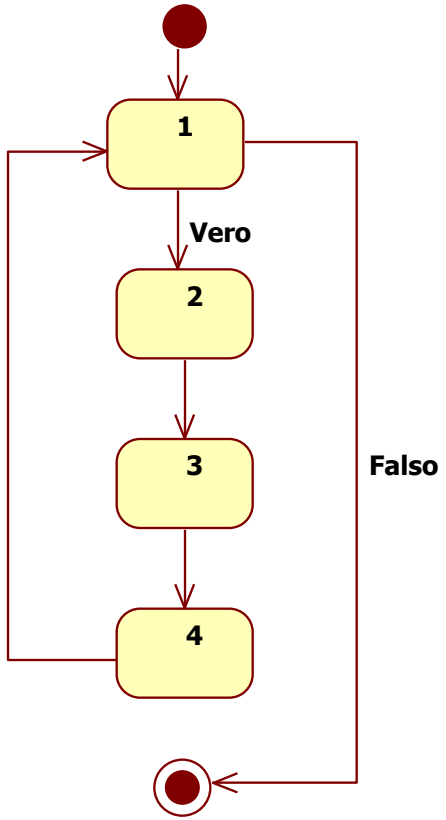
```
1.  while
    (decisione)
2.  {
3.    Blocco
4.  }
```



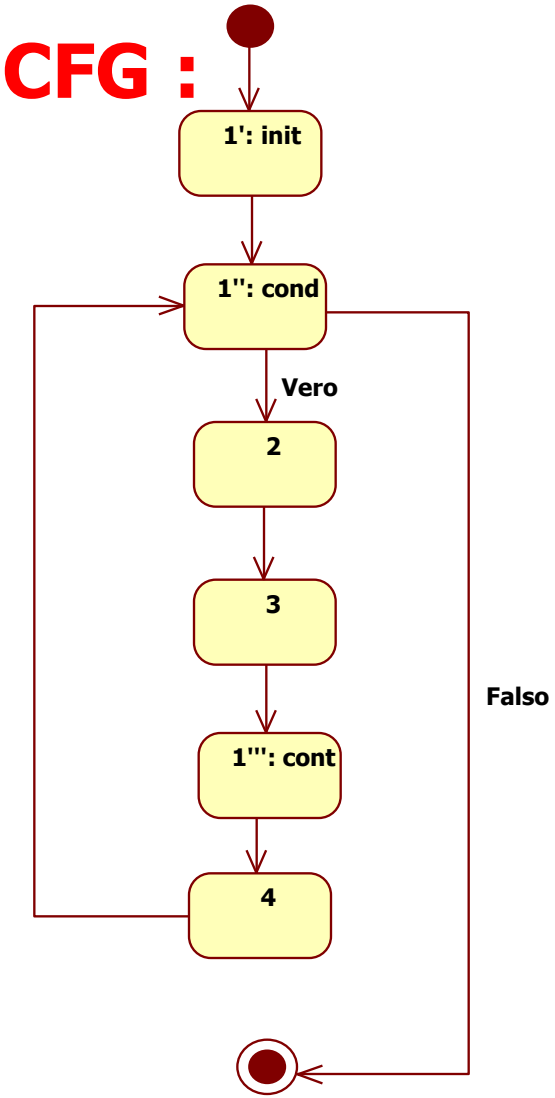

```
1.  do
2.  {
3.    Blocco
4.  }
5.  while
    (decisione
    )
```



Control structures on the CFG : For




```
1.   for (init;decisione;cont)
2.   {
3.     Blocco
4.   }
```



The CFG on the right expresses the semantics of the for loop at executable code level. In fact, each for loop corresponds to an initial init, a decision at the beginning of each loop (as in a while) and a cont continuation code (for example i ++) to repeat before starting the loop again

Other definitions

- A path on a CFG is a route from the starting node to the final node (compliant with edge arrows)
  - A path on the CFG corresponds to the execution of a valid test case
    - *Test cases that does not terminates their execution are not considered valid test cases*
- A node of the CFG is covered by a test case if the corresponding path include that node
- An edge of the CFG is covered by a test case if the corresponding path include that edge

Coverage criteria

- CFG Coverage criteria represent objectives of a testing activity
- CFG Coverage measures represent effectiveness measures for test suites

Coverage criteria

- **Statement (node) coverage**
  - **Requires that each CFG node be run at least once during testing;**
    - ***it is a weak coverage criterion, which does not ensure coverage of both the true and false branches of a predicate.***

Coverage criteria

- **Decision coverage (branch coverage or edge coverage)**
  - **It requires that each edge of the CFG be included in at least one test case path**
    - ***In this case, there is at least a test case in which the decision is true and at least a test case where the decision is false***
      - **maybe the same test case, if there are loops**

Decisions vs Conditions

- A decision is a Boolean predicate
  - It may correspond either on a single clause or to a complex Boolean expression
- A decision may be a composition of clauses connected by logic (Boolean) operators
- A condition is a clause in a decision predicate

((a<0 && b>0) || c<0) is a decision

a<0, b>0, c<0 are the corresponding conditions

Coverage criteria

- **Condition coverage**
  - It requires that for each condition there is at least a test case in which the condition is true and at least a test case where the condition is false
    - maybe the same test case, if there are loops

- **Properties:**
  - Condition coverage does not imply decision coverage
  - Decision coverage does not imply condition coverage

Consideration

- At source code level, decisions may be composed of one or more conditions
- At executable code level, there are only decisions with a single clause (e.g. BRA, BZ, BNZ)
- A decision at source code level is translated in a possibly long sequence of statements at executable code level
- Consequence:
  - At executable code level, decisions coincides with conditions

Coverage criteria

- Condition combination coverage
  - It requires that for each decision (composed of one or more conditions) there is at least a test case covering each possible combination of the Boolean values of the conditions

- The number of condition combination for each decision is 2^n, where n is the number of clauses composing the condition

- Property:
  - The condition combination coverage implies the decision coverage and also the condition coverage

Example

- if (x>0 && y>0) ...
  - TS1={(x=2, y=-1), (x=-1, y=5)} covers all the conditions but not all the decisions
  - TS2={(x=2, y=1), (x=-1, y=-55)} covers all the conditions and all the decisions but not all the condition combinations
  - TS3={(x=2, y=1), (x=2, y=-1), (x=-2, y=1)(x=-1, y=-55)} covers all the condition combinations (and that implies that it covers all the decisions and all the conditions)

Example

```
int x;
{    if ((x>=0) && (x<= 200))
  check= true;
      else check = false;
        }
```

TS={{x=5}, {x=-5} } cover both the decisions (T, F)

TS1={{x= 3}, {x=-1}, {x=210}} covers all the conditions (x>=0 is true or false and x<= 200 is true or false at least once) but only three different condition combinations (TT, FT, TF). The fourth combination (FF) is infeasible

TS2={{x=-4};{x=300}} covers all the conditions (x>=0 is true or false and x<= 200 is true or false at least once) but only the F decision

JUnit

Scrittura dei test di unità

- Il testing a livello di unità dei comportamenti di una classe dovrebbe essere progettato ed eseguito dallo sviluppatore della classe, contemporaneamente allo sviluppo stesso della classe

- **Vantaggi:**
  - Lo sviluppatore conosce esattamente le responsabilità della classe che ha sviluppato e i risultati che da essa si attende
  - Lo sviluppatore conosce esattamente come si accede alla classe, ad esempio:
    - Quali precondizioni devono essere poste prima di poter eseguire un caso di test;
    - Quali postcondizioni sui valori dello stato degli oggetti devono verificarsi
- **Svantaggi**:
  - Lo sviluppatore tende a difendere il suo lavoro … troverà meno errori di quanto possa fare un tester!

Testing Automation

- L'automatizzazione dell'esecuzione dei casi di test porta innumerevoli vantaggi:
  - **Tempo risparmiato** (nell'esecuzione dei test)
  - **Affidabilità dei test** (non c'é rischio di errore umano nell'esecuzione dei test)
    - Si può migliorare l'efficacia a scapito dell'efficienza sfruttando il fatto che l'esecuzione dei test è poco costosa
  - **Riuso** (**parziale**) dei test a seguito di modifiche nella classe

Testing basato su main

- Scrivere un metodo di prova (“main”) in ogni classe contenente del codice in grado di testare I suoi comportamenti
  - Problemi
    - Tale codice verrà distribuito anche nel prodotto finale, appesantendolo
    - Come strutturare i test case? Come eseguirli separatamente?
    - La classe potrebbe già avere un metodo *main*

- Per tutti questi motivi, cerchiamo un approccio sistematico
  - Che separi il codice di test da quello della classe
  - Che supporti la strutturazione dei casi di test in test suite
  - Che fornisca un output separato dall’output dovuto all’esecuzione della classe

Famiglia X-Unit

- La soluzione alle problematiche precedenti é data dai framework della famiglia X-Unit:
  - **JUnit** (Java)
    - È il capostipite; fu sviluppato originariamente da Erich Gamma and Kent Beck
    - Si utilizzano diverse version: JUnit 3, JUnit 4 e JUnit 5. In questo corso si adotterà la pi generale **JUnit 4**
  - CppUnit (C++)
  - csUnit (C#)
  - NUnit (.NET framework)
  - HttpUnit, HTMLUnit (Web Application)
  - JSUnit (JavaScript)
  - PHPUnit (Php)

JUnit

- JUnit é un framework di unit testing per il linguaggio di programmazione Java.
- Esso include una classe di test detta **TestRunner** che :
  - funge da main (può essere chiamata dall'esterno)
  - Esplora automaticamente le classi di test individuando tutti I loro component
  - Li esegue secondo un preciso schema
  - Genera I risultati dei test
- Tutti gli IDE (incluso IntelliJ) hanno nativamente dei plug-in che consentono di eseguire I test e visualizzare I risultati

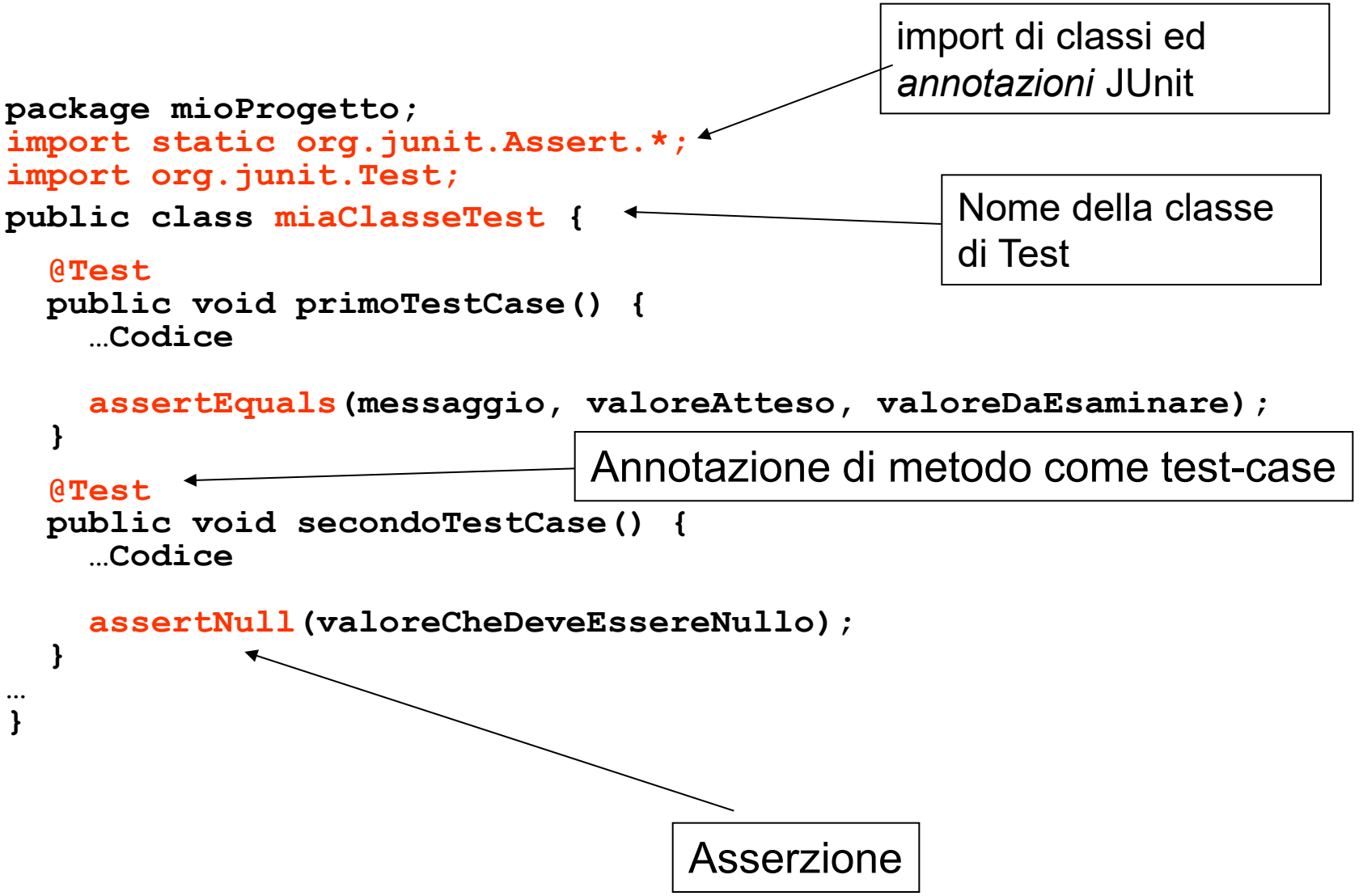
Componenti di un test JUnit 4

- Una classe di test, contenente:
  - Un metodo detto di ***setup***, *preceduto dall'annotazione* ***@Before*** che viene eseguito prima dell'esecuzione di ogni test
    - Utile per settare precondizioni comuni a più di un caso di test
    - Nel metodo di setup bisogna anche verificare le precondizioni
  - Un metodo detto di ***teardown***, *preceduto dall'annotazione* ***@After*** che viene eseguito dopo ogni caso di test
    - Utile per resettare le postcondizioni
    - Nel metodo di teardown bisogna anche verificare le eventuali postcondizioni
  - Un metodo per ogni caso di test, ognuno preceduto dall'annotazione **@Test**
    - In ogni metodo di test bisogno verificare che il risultato del test sia quello atteso

Struttura di un metodo di test

- **Inizializzazione precondizioni**
  - Limitatamente alle precondizioni tipiche del singolo caso di test, le altre dovrebbero essere nel *setup*
- **Inserimento valori di input**
  - Tramite chiamate a metodi set oppure tramite assegnazione di valori ad attributi pubblici
- **Codice di test**
  - Esecuzione del metodo da testare con gli eventuali parametri relativi a quel caso di test
- **Valutazione delle asserzioni**
  - Controllo di espressioni booleane (*asserzioni*) che risultano **FALSE** quando il test è stato in grado di rilevare un malfunzionamento, ovvero se i dati di uscita e/o le post condizioni riscontrati sono diversi da quelli attesi

JUnit 4: Classi di Test




```
package mioProgetto;
import static org.junit.Assert.*;
import org.junit.Test;
public class miaClasseTest {

  @Test
  public void primoTestCase() {
    …Codice

    assertEquals(messaggio, valoreAtteso, valoreDaEsaminare);
  }

  @Test
  public void secondoTestCase() {
    …Codice

    assertNull(valoreCheDeveEssereNullo);
  }
…
}
```

JUnit: Osservazioni

- Tutte le classi di test che scriveremo avranno questa struttura
- Ovviamente le classi di test vanno progettate sulla base delle peculiarità della classe testata

import static org.junit.Assert.*;

- Serve per importare metodi (statici) e annotazioni del framework Junit

@Test è un'*annotazione* per marcare i metodi che si considerano di Test

- Non è (più) necessario ma è buona norma usare '**test**' come prefisso del nome dei metodi di test (obbligatorio in Junit 3)

```
@Test
public void testMetodoDaTestare() {
…
}
```

Asserzioni

- ***Asserzione***
  - affermazione che può essere vera o falsa

- I risultati attesi sono documentati con delle *asserzioni* esplicite, non con delle stampe che comunque richiedono dispendiose ispezioni visuali dei risultati
- Se l'asserzione è
  - ***Vera*** → il test è passato senza rilevare malfuzionamenti (**Test Passed**)
  - ***Falsa*** → il test è fallito ed il codice testato non si comporta come atteso, quindi c'è un errore a tempo dinamico (**Test Failed**)

- Le asserzioni sono utilizzate sia per verificare gli oracoli che le postcondizioni
  - Le asserzioni potrebbero essere utilizzate anche per verificare le precondizioni: se la precondizione fallisce, il test non viene proprio eseguito (e viene riportato come fallito)

Asserzioni

- Se una asserzione non è vera il test-case fallisce
  - **assertNull()**: afferma che il suo argomento è nullo (fallisce se non lo è)
  - **assertEquals()**: afferma che il suo secondo argormento è **equals()** al primo argomento, ovvero al valore atteso
  - molte altre varianti
    - **assertNotNull()**
    - **assertTrue()**
    - **assertFalse()**
    - **assertSame()**
    - **…**

assertEquals()

- **assertEquals(Object expected, Object actual)**
  - Va a buon fine se e solo se **expected.equals(actual**) restituendo **True**

    expected è il valore atteso

    actual è il valore effettivamente rilevato

- **assertEquals(String message, Object expected, Object actual)**
  - In questa variante si specifica un messaggio che il *runner* stampa in caso di fallimento dell'asserzione: molto utile per localizzare immediatamente l'asserzione che causa il fallimento di un test-case ed avere i primi messaggi diagnostici

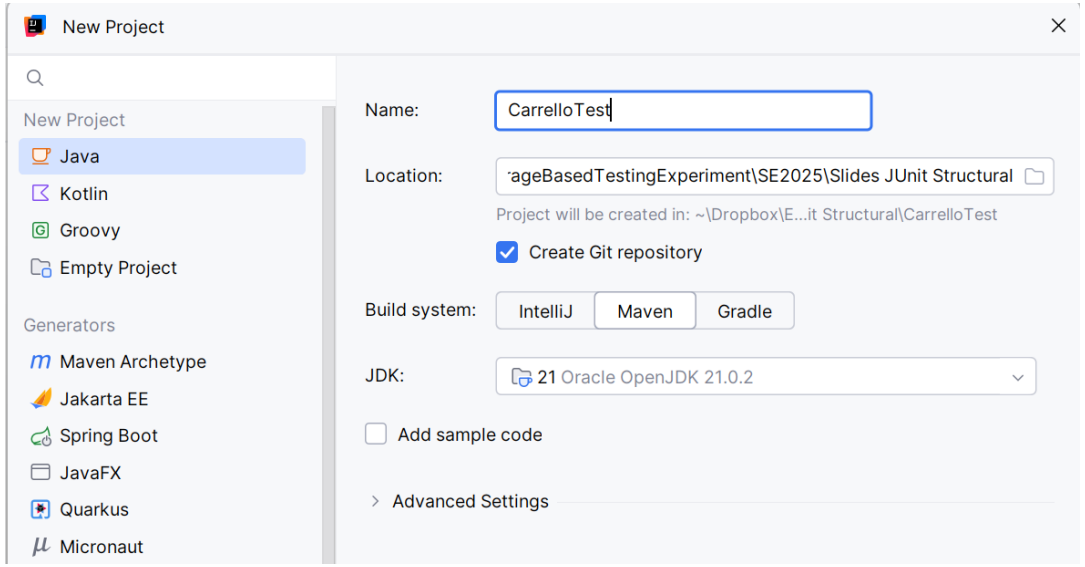
Test di unità in pratica

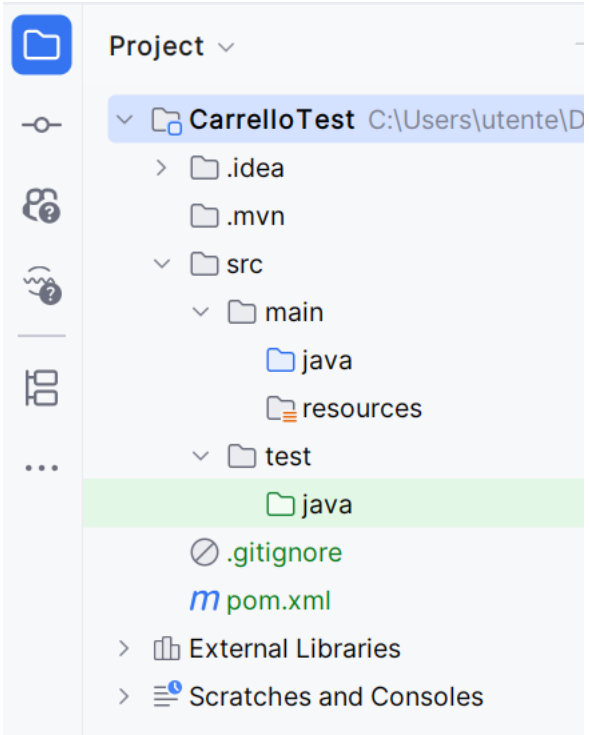
```
//Test del Costruttore dell'oggetto
public void testMetodoDaTestare() {
   miaClasse mioOggetto = new miaClasse("par1");
   assertEquals("par1", mioOggetto.getNome());
}
```

- mettere un "frammento" del sistema in uno stato noto
  - il frammento comprende un solo oggetto **mioOggetto**
- inviare una serie di messaggi noti
  - nell'esempio solo la costruzione dell'oggetto, in generale ci potrebbero essere altre invocazioni di metodo sullo stesso
- controllare che alla fine il sistema si trovi nello stato atteso
  - tramite le asserzioni si controlla che il nome ed il parametro del comando siano quelli attesi

Tutorial : JUnit in IntelliJ

- Il progetto è già impostato per la presenza dei test se scegliamo di crearlo con l'opzione Maven





Classi di test

- Le classi di test possono essere create nel folder test/java esattamente come si possono creare le classi normali
  - Potenzialmente le classi possono trovarsi anche in altri folder, eventualmente anche nello stesso folder delle classi di sviluppo
  - La separazione tra classi di sviluppo e classi di test ha effetti benefici nella divisione dei compiti di lavoro e nella compilazione ed esecuzione separata delle varie parti

Esempio

- Una classe per la gestione del carrello di un sito di E-Commerce
  - Nome della classe: **Cart**
  - Metodi implementati:
    - **`getTotal()`** - restituisce il numero di elementi nel carrello
    - **`insertProduct(String nome)`** – inserisce il prodotto nel carrello se c'è ancora spazio (il carrello può contenere al massimo tre prodotti) e, nel caso, incrementa di uno il numero di elementi nel carrello
    - **`removeProduct(String nome)`** – se il prodottoindicato è nel carrello lo elimina e decrementa di uno il numero di elementi nel carrello
    - **`removeAllProducts()`** – rimuove tutti i prodotti e azzera il numero di elementi nel carrello
    - **`isEmpty()`** – restituisce true se il carrello è vuoto

```
import java.util.ArrayList;

public class Cart {
   private int total;
   private ArrayList<String> product;

   public Cart() {
     product = new ArrayList<String>();
     total=0;
   }
   public int getTotal() {return total;   }
   public void insertProduct(String nome) {
     if (total<3) {
        product.add(nome);
        total++;
        }
   }
   public void removeProduct(String nome) {
     if (product.contains(nome)) {
        product.remove(nome);
        total--;
     }
   }
   public void removeAllProducts() {
     product.clear();
     total = 0;
   }
   public boolean isEmpty() { return total==0;   }
}
```

Scriviamo i casi di test

- Obiettivo: eseguire tutti i metodi e tutte le righe di codice
- Prima di tutto abbiamo bisogno di un oggetto Cart (carrello)
  - Siccome ogni test avrà bisogno di un carrello valido, allora ha senso che lo inizializziamo nel metodo setup e lo distruggiamo nel metodo tearDown

- Le asserzioni verificano che l'oggetto sia stato effettivamente istanziato / annullato

```
import org.junit.After;
import org.junit.Before;
import org.junit.Test;

import static org.junit.Assert.*;

public class CartTest {
   private Cart cart;

   @Before
   public void setUp() throws Exception {
      cart = new Cart();
      assertFalse(cart==null);
   }

   @After
   public void tearDown() throws Exception {
      cart = null;
      assertTrue(cart==null);
   }
}
```

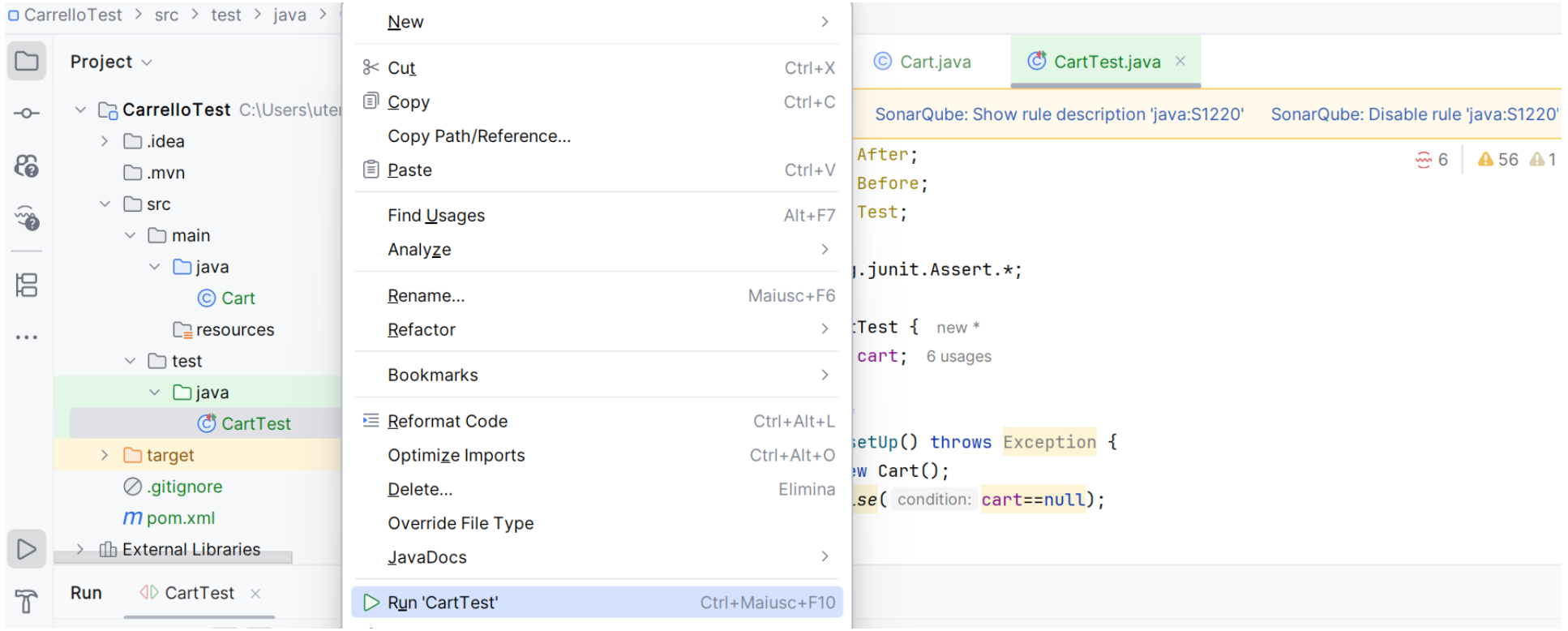
Primo test

- Vogliamo provare ad inserire un elemento nel carrello e controllare che il totale sia stato effettivamente incrementato

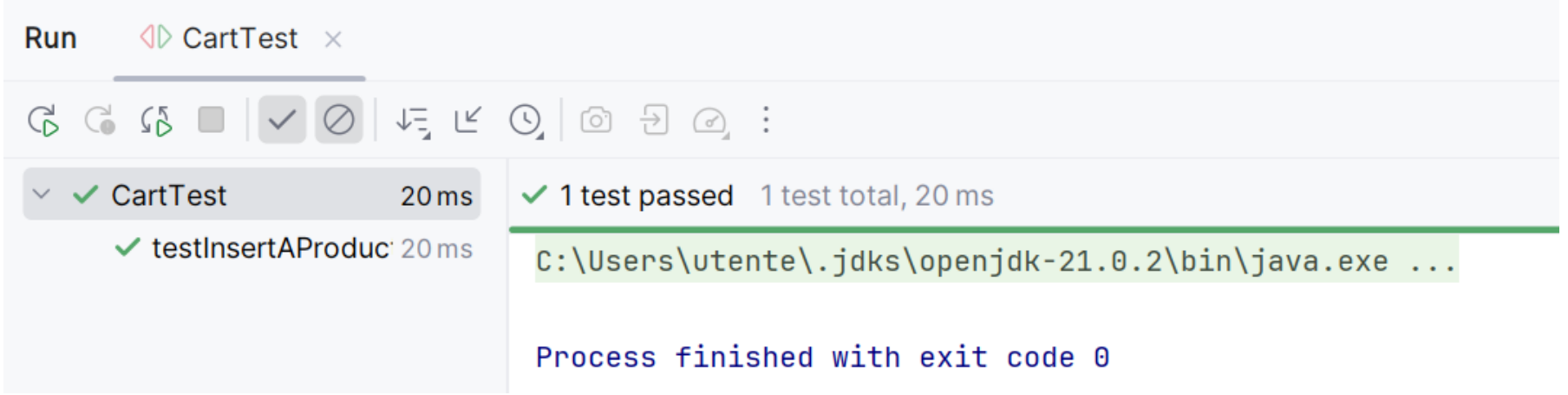
```
@Test
public void testInsertAProduct() {
   cart.insertProduct("prodotto1");
   assertEquals("Errore nel totale", 1, cart.getTotal());
}
```

Esecuzione del primo caso di test

- Si può eseguire anche dal simbolo che compare  dice



Risultato del test

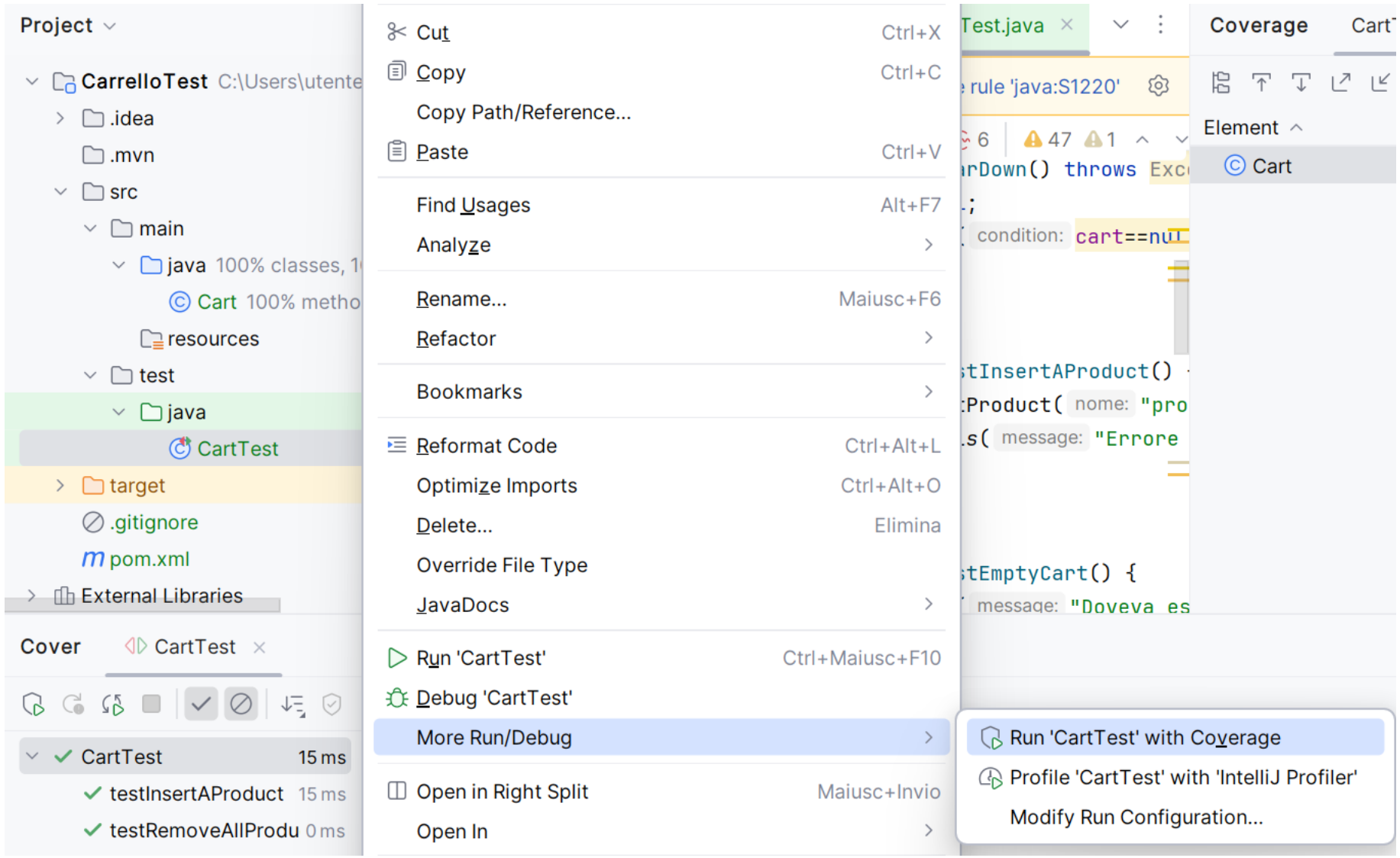


Altri test

```
// controlla che cart nasca vuoto
    @Test
    public void testEmptyCart() {
        assertTrue("Doveva essere
vuoto",cart.isEmpty());
    }

//inserisce, poi rimuove
@Test
    public void testRemoveOneProduct() {
        cart.insertProduct("prodotto1");
        cart.removeProduct("prodotto1");
        assertEquals("Doveva essere rimosso", 0,
cart.getTotal());
    }
```

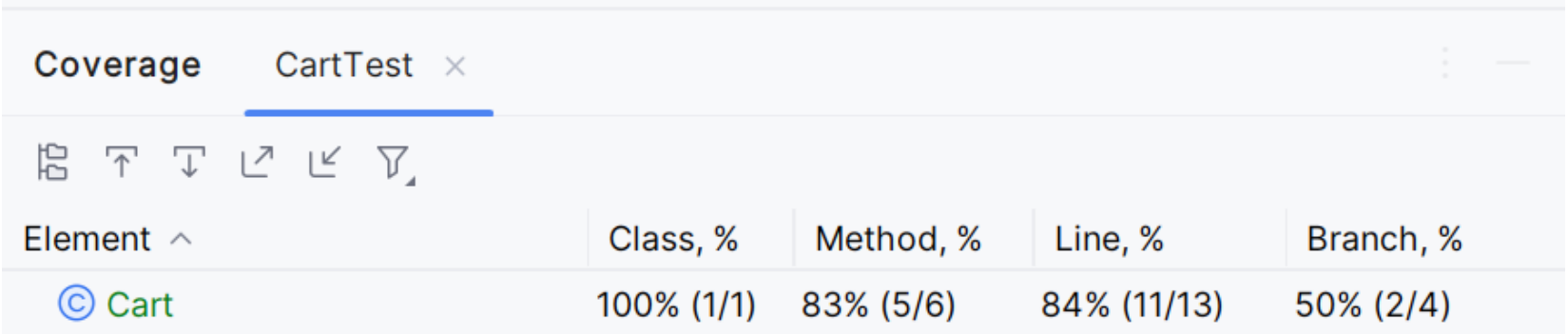
Copertura

- Quanta parte del codice è stata **coperta** dai test effettuati?
  - C’è una complessa teoria dietro questa definizione ma è molto semplice avere una prima valutazione
  - IntelliJ si integra con uno strumento di calcolo della copertura (JaCoCo) che può essere avviato con l’opzione **Run Test with Coverage …**


Project
CarrelloTest C:\Users\utente
.idea
.mvn
src
main
java 100% classes, 1
Cart 100% metho
resources
test
java
CartTest
target
.gitignore
pom.xml
External Libraries
Cover
CartTest
CartTest 15 ms
testInsertAProduct 15 ms
testRemoveAllProdu 0 ms
Cut Ctrl+X
Copy Ctrl+C
Copy Path/Reference...
Paste Ctrl+V
Find Usages Alt+F7
Analyze
Rename... Maiusc+F6
Refactor
Bookmarks
Reformat Code Ctrl+Alt+L
Optimize Imports Ctrl+Alt+O
Delete... Elimina
Override File Type
JavaDocs
Run 'CartTest' Ctrl+Maiusc+F10
Debug 'CartTest'
More Run/Debug
Open in Right Split Maiusc+Invio
Open In
Test.java
rule 'java:S1220'
6 47 1
rDown() throws Exc
condition: cart==nul
tInsertAProduct()
Product(nome: "pro
s(message: "Errore
tEmptyCart() {
message: "Doveva es
Coverage
Cart
Element
Cart
Run 'CartTest' with Coverage
Profile 'CartTest' with 'IntelliJ Profiler'
Modify Run Configuration...

Risultati di copertura

- La copertura è visualizzata sia come risultati complessivi che colorando opportunamente il codice



Jacoco behaviour

- At bytecode level, the coverage of each block is boolean (covered / not covered)
- The projection of bytecode coverage on the source code generates fractionary values of coverage for each line of code:
  - 1: if all the bytecode blocks generated by the line of code have been covered
  - Fractionary, between 0 and 1, the only some of the blocks have been covered
  - 0: if no blocks have been covered
  - Of course, no coverage is measured for all the lines that do not generate bytecode blocks (e.g. comments, blank lines, parentheses, etc.)

Copertura in dettaglio

- Il verde indica le righe eseguite
- Il rosso le righe non eseguite
- Il giallo le righe parzialmente eseguite (la 19 è stata provata solo quando era vera)

```
7       public Cart() {  1 usage  new *
8           product = new ArrayList<String>();
9           total=0;
10      }
11      public int getTotal() {return total;     }  4 usages
12      public void insertProduct(String nome) {  7 usages  n
13          if (total<3){
14              product.add(nome);
15              total++;
16          }
17      }
18      public void removeProduct(String nome) {  3 usages  n
19          if (product.contains(nome)) {
20              product.remove(nome);
21              total--;
22          }
23      }
24      public void removeAllProducts() {  1 usage  new *
25          product.clear();
26          total = 0;
27      }
28      public boolean isEmpty() {  return total==0;     }
29  }
```

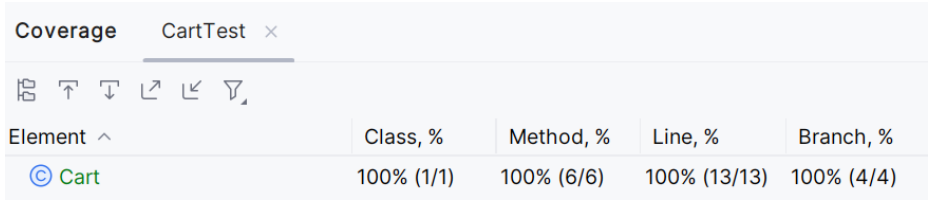
Coverage-based testing

- L’osservazione del codice non eseguito dai test ci può portare alla scrittura di nuovi casi di test che possano portare a eseguire tutto il codice disponibile
  - Nel nostro esempio ci rimangono da eseguire il caso di carrello pieno, l’eliminazione di un prodotto non presente nel carrello e l’eliminazione di tutti i prodotti dal carrello

Test da aggiungere

```
@Test
public void testRemoveAllProducts() {
   cart.insertProduct("prodotto1");
   cart.insertProduct("prodotto2");
   cart.removeAllProducts();
   assertTrue("Doveva essere vuoto",
cart.isEmpty());
}
```

```
@Test
public void testRemoveProductNotInCart() {
   cart.insertProduct("prodotto1");
   cart.removeProduct("prodotto2");
   assertEquals("Non doveva essere rimosso", 1,
cart.getTotal());
}
```




```
@Test
public void testInsertMultipleProducts() {
   cart.insertProduct("prodotto1");
   cart.insertProduct("prodotto1");
   cart.removeProduct("prodotto1");
   assertEquals("Errore nel totale", 1,
cart.getTotal());
}
```

```
@Test
public void testInsertMoreThanThreeProducts() {
   cart.insertProduct("prodotto1");
   cart.insertProduct("prodotto2");
   cart.insertProduct("prodotto3");
   cart.insertProduct("prodotto4");
   assertEquals("Errore nel totale", 3,
cart.getTotal());
}
```

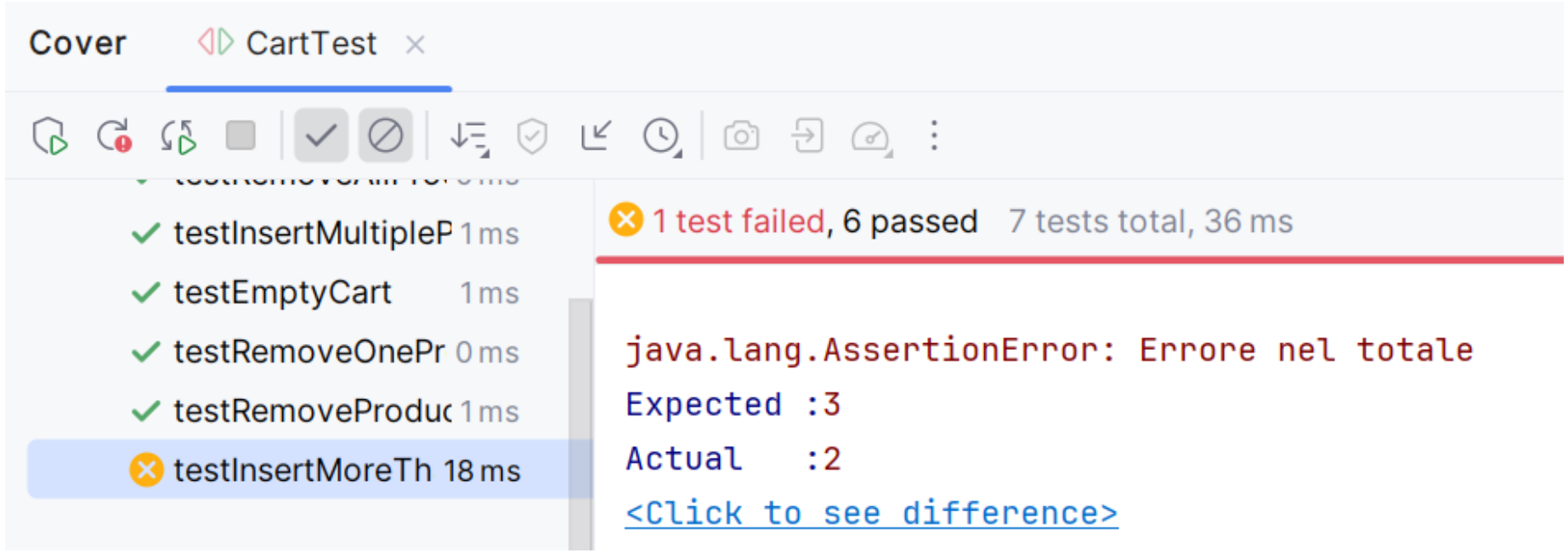
Trovare un difetto

- Per provare a trovare un difetto, immaginiamo di aggiungerlo noi
- Immaginiamo che il limite di dimensione del carrello sia stato impostato a 2 anziché a 3

```
public void insertProduct(String nome) {
        if (total<2){
                product.add(nome);
                total++;
        }
```

- I nostri test saranno in grado di accorgersi di questo difetto?

Output dei test

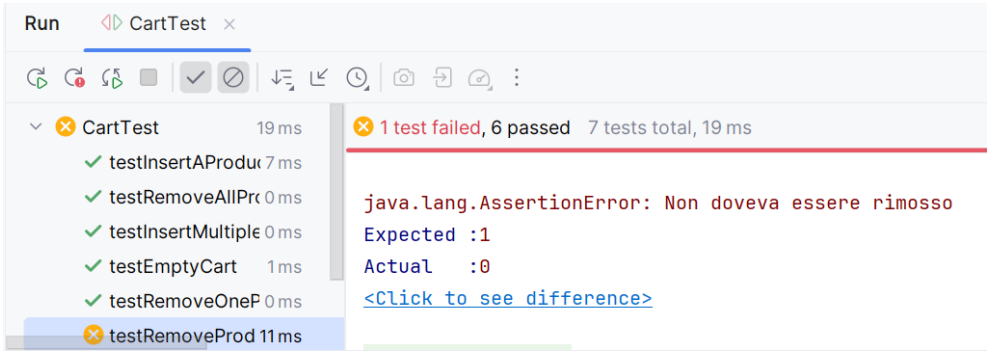


Altro errore

- Immaginiamo che total - - sia stato messo erroneamente fuori dall'if on RemoveProduct

```
public void removeProduct(String nome) {
   if (product.contains(nome)) {
      product.remove(nome);
   }
   total--;
}
```

- Questo errore è riconoscibile dai nostri test?



Isolation Testing and Integration Testing

Isolation Testing

Unit testing could only be applied to completely detached pieces of software:

- detached from the rest of the software
  (other classes, libraries, ...)
- detached from other software
  (operating system, ...)
- detached from other resources
  (database, file, network, ...)

Unit Test Limitations

You cannot test a module in isolation if it:

- Communicates with the database
- Communicates over the network
- Communicates with other modules not yet tested
- edits databases/files or other data sources
- cannot be launched in parallel with other test cases

Isolation Testing

Solution supporting isolation testing

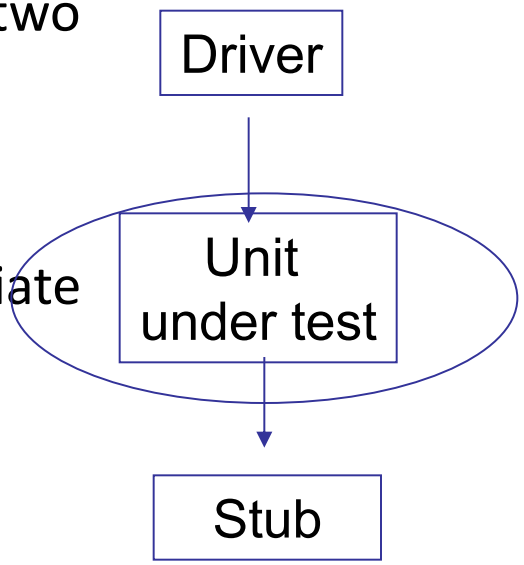
Two possible conceptual solutions:

1) Assume the correctness of all modules called by the module under test and the validity of all resources accessed by it

2) Replace all called modules and accessed resources with fictitious, simplified versions, the correctness of which can be imposed by construction

Testing a "non-terminal" module: drivers and stubs

To test a non-terminal module, you need to build two types of modules:



Drivers

- invoke the unit under test, sending it appropriate values, relative to the test case

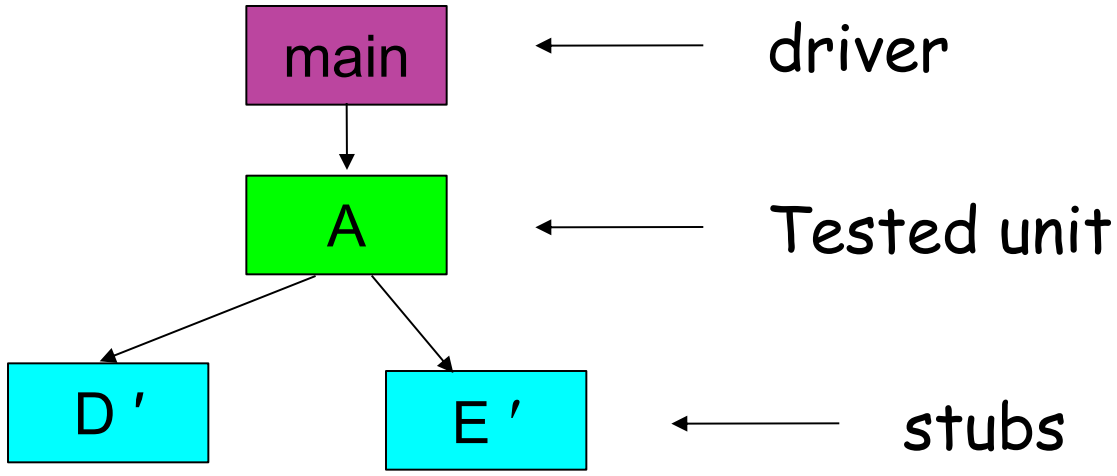
•Stubs

- are invoked by the unit under test;
- emulate the operation of the called function with respect to the required test case (taking into account the specifications of the called function)
  - When the called function is built and tested, you will replace the stub with the function itself

Stub and Driver

Tested units need to be called (by Drivers)

Called modules must be replaced by others (Stub)



Driver

- A driver module must completely replace the caller module(s) of the module to be tested
  - A TestCase method under JUnit can implement a driver
- The driver module must:
  - Set all the values of the resources and data sources used by the module to be tested
  - In object-oriented languages, construct the object whose method is being tested
  - Start the method you want to test

Stub

- A stub is a fictitious function whose correctness is true by hypothesis
  - Example: if we are testing a function prod_scal(v1,v2) that invokes a function product(a,b) but we have not yet realized that function
  - In the driver method we write code to run some test cases
    - For example, we call **prod_scal([2,4],[4,7])**
  - The stub method can be written like this:

```
int product(int a, int b){
    if (a==2 && b==4) return 8;
    if (a==4 && b==7) return 28;
    }
```

  - The correctness of this stub method is given by hypothesis
  - Obviously, in order to set up this testing, it will be necessary to have precise information on the internal behavior required of the module to be tested.

Stub

- The term Stub is used, more generally, to indicate a fictitious method, not yet implemented or whose implementation is, intentionally, incomplete.
  - Often stubs are put in code simply as a reminder of methods yet to be implemented or to allow the code to be compiled as soon as possible
- The stub has the task of reproducing the behavior of the module that it replaces only in the test cases provided by the drivers made
  - The stub can be written on the basis of a 'black box' knowledge of the module to be emulated
- Stubs allow you to test a module before the modules on which it depends have been built

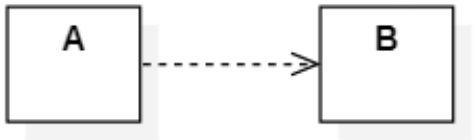
Stub

- A Stub can effectively replace a function/method
  - Each test case must correspond to a different instance of the stub (or a different branch of the stub)

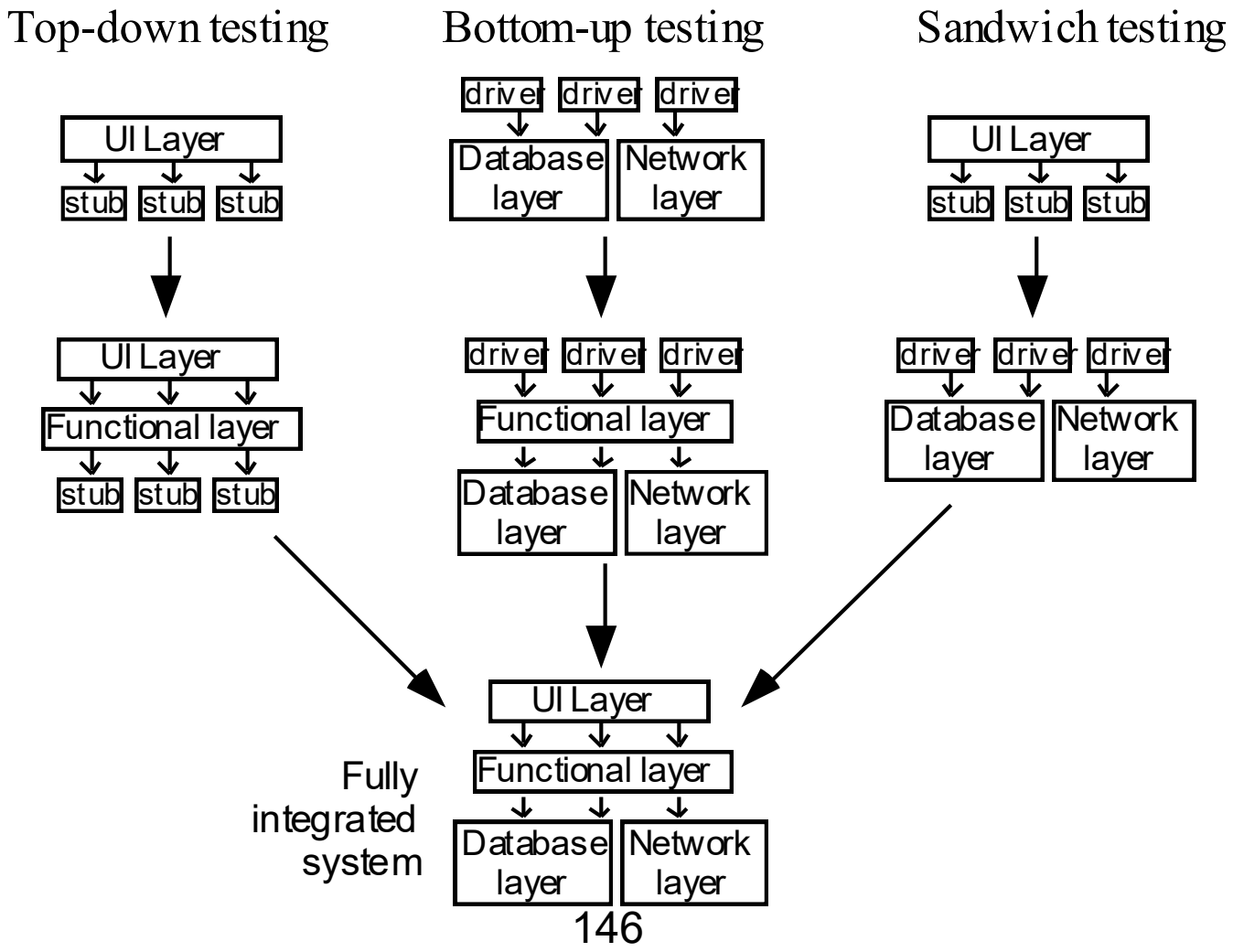
- A Stub does not effectively replace a class
  - Cannot handle the state (attribute values) of a class between two stub calls
  - Cannot handle testing a sequence of interactions

Integration testing for two units

- Once all units have been tested, integration testing is required to assess whether they are working properly as a whole.
  - If a calls b, and both a and b have passed unit tests, the set of a and b may not satisfy all the required tests of a
    - Possible problems could be related, for example, to combinations of the inputs of b that those who tested b did not predict but which may appear as possible call values in a
  - To test a and b (with a calling b) you may only need to use tests designed for a
    - In practice, you replace, in tests designed for a, the real module b instead of its stub (or mock)

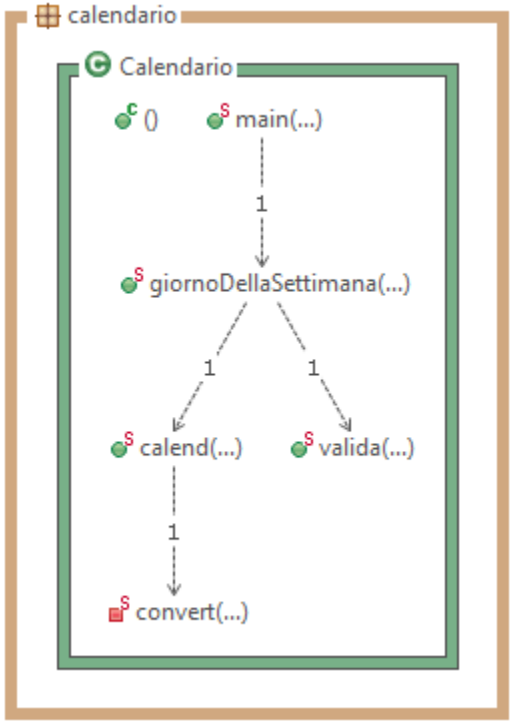


Integration Testing Strategies



JUnit integration testing example

- Let's consider a single Calendar class with the following methods:
  - main reads from command line day, month (string) and year
  - giornoDellaSettimana converts the input month to an integer, checks if the date is valid and calls calend if it is valid
  - valida check if the date is valid
  - calend calculates the day of the week (integer)
  - convert converts the resulting day of the week to string



Source code: giornoDellaSettimana

```
public static String giornoDellaSettimana(int d, String ms, int a)
{
int m=0;
if (ms.equals("gennaio"))
m=1;
else if (ms.equals("febbraio"))
m=2;
else if (ms.equals("marzo"))
m=3;
else if (ms.equals("aprile"))
m=4;
else if (ms.equals("maggio"))
m=5;
else if (ms.equals("giugno"))
m=6;
else if (ms.equals("luglio"))
m=7;
else if (ms.equals("agosto"))
m=8;
else if (ms.equals("settembre"))
m=9;
else if (ms.equals("ottobre"))
m=10;
else if (ms.equals("novembre"))
m=11;
else if (ms.equals("dicembre"))
m=12;
if ((m>0) && valida(d,m,a))
return calend(d,m,a);
else
return "Errore";
}
```

Source code: valida

```
public static boolean valida(int d, int m, int a) {
if (d<1 || d>31 || m==0 || a<=1582)
return false;
Boolean bisestile= (a%4==0);
if (bisestile && a%100==0 && a%400!=0)
bisestile=false;
if ((m==2 && d>29)||(m==2 && d==29 &&
!bisestile))
return false;
if ((m==4 || m==6 || m==9 || m==11) && d>30)
return false;
return true;

}
```

Source code: calend

```
public static String calend(int d, int m, int a)
{
if (m<=2)
 {
  m = m + 12;
  a--;
 };
 int f1 = a / 4;
 int f2 = a / 100;
 int f3 = a / 400;
 int f4 = (int) (2 * m + (.6 * (m + 1)));
 int f5 = a + d + 1;
 int x = f1 - f2 + f3 + f4 + f5;
 int k = x / 7;
 int n = x - k * 7;
 return convert(n);
}
```

Integration Testing

Source code: convert

```
public static String convert(int n) {
 if (n==1)
 return "Lunedi";
 else if (n==2)
 return "Martedi";
 else if (n==3)
 return "Mercoledi";
 else if (n==4)
 return "Giovedi";
 else if (n==5)
 return "Venerdi";
 else if (n==6)
 return "Sabato";
 else if (n==0)
 return "Domenica";
 else return "Errore";
}
```

Integration Testing

Source code: main

```
public static String main(String[] args) {
if (args.length==3){
int giorno=Integer.parseInt(args[0]);
String mese=args[1];
int anno=Integer.parseInt(args[2]);
String giornoDS=new String(giornoDellaSettimana(giorno,mese,anno));
System.out.println(giornoDS);
return giornoDS;
}
else
return "";
}
```

Integration Testing

Bottom-up example 1/2

- Let's start by testing convert

```
@Test
 public void testConvert1() {
 assertEquals("Domenica",Calendario.convert(0));
 }
 …
```

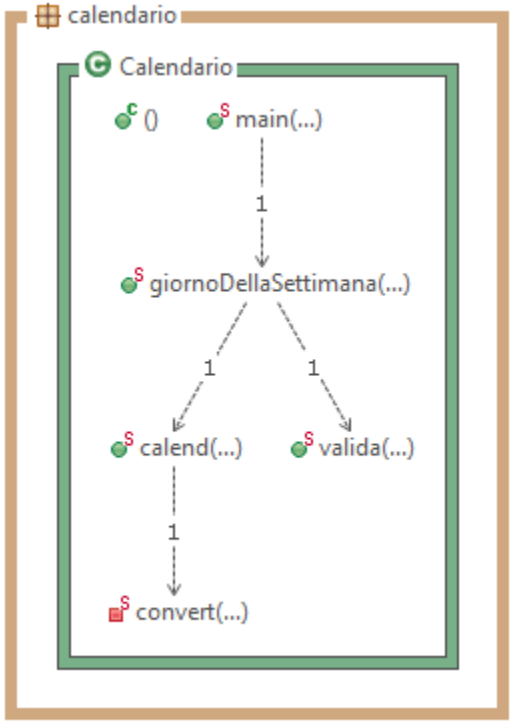
- Let's continue with calend

```
@Test
public void testCalend1() {
assertEquals("Domenica",Calendario.calend(24
, 4, 2011));
}
```

…

- Then valida:

```
 @Test
 public void testValida1() {
 assertTrue(Calendario.valida(24, 4, 2011));
 }
 …
```



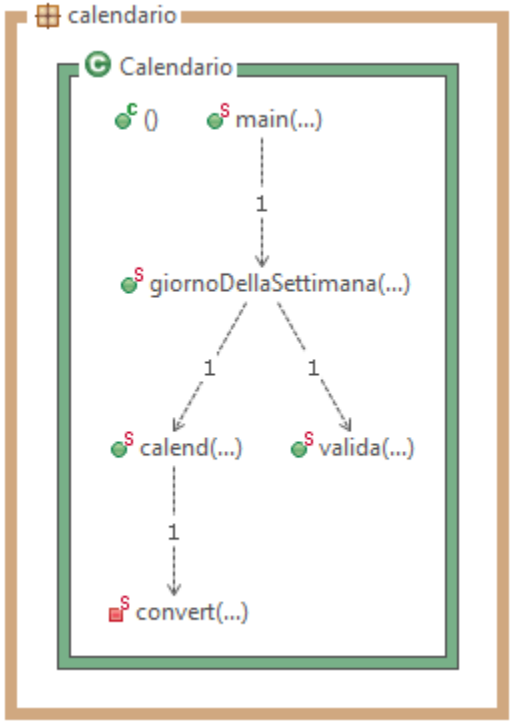
Bottom-up example 2/2

- Then giornoDellaSettimana:

```
@Test
public void testGiornoDellaSettimana1() {
assertEquals("Domenica",Calendario.giornoDellaSettimana(24,
   "aprile", 2011));
}
…
```

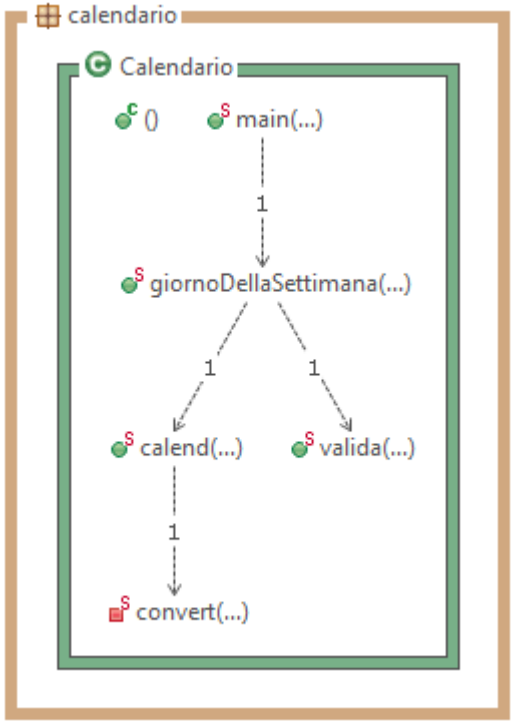
- Finally, main:

```
@Test
public void testMain1() {
String a[]=new String[3];
a[0]="24";
a[1]="aprile";
a[2]="2011";
assertEquals("Domenica",Calendario.main(a)),
}
…
```



Top-down example 1/2

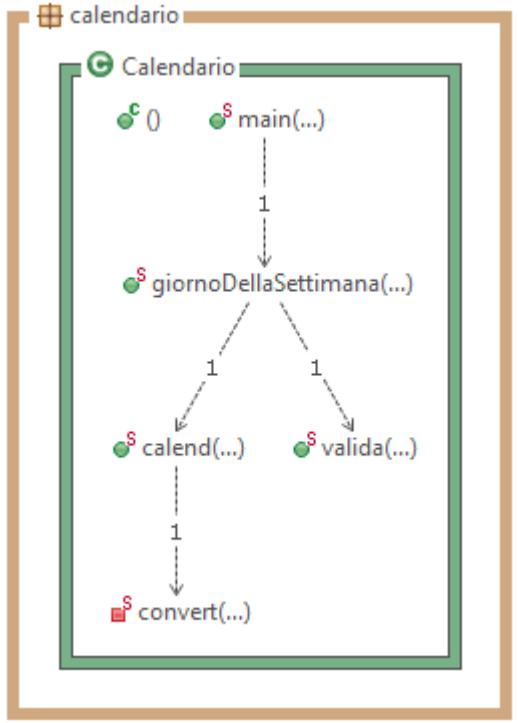
- To test the main class we only need a stub for giornoDellaSettimana.
-
```
public static String giornoDellaSettimana(int d, String ms, int a){
 //STUB
 if (d==24 && ms.equals("aprile") && a==2011)
    return "Domenica"; //TC1
 else if (d==32 && ms.equals("aprile") &&
    a==2011) return ""; //TC2
 return "";
 }
```
- We can then perform the same tests designed for the main in the bottom-up strategy



Esempio di testing top-down 2/2

- To test the class giornoDellaSettimana we need stubs both for calend and for valida

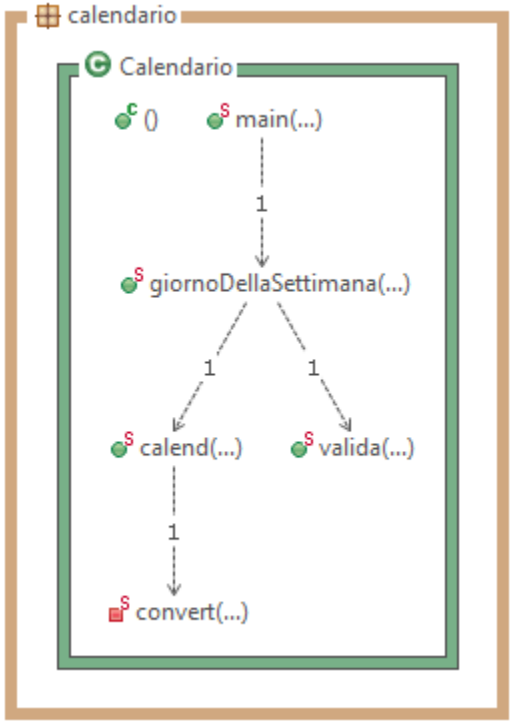
```
public static boolean valida(int d, int m, int
   a) {
//STUB
if (d==24 && m==4 && a==2011) return true;
else if (d==29 && m==2 && a==2012) return true;
else if (d==32 && m==4 && a==2011) return
   false;
return false;
}


public static String calend(int d, int m, int
   a){
// STUB
if (d==24 && m==4 && a==2011) return
   "Domenica";
else if (d==29 && m==2 && a==2012) return
   "Lunedi";
else return "";
}
```



- At this point we reuse the calend tests, and so on

Other strategies: testing in isolation

- If, for example, the calend class had been the first to be realized
  - For example for prototyping reasons, since it is the only class with complexity deriving from astronomy knowledge
- we need:
  - a driver with the behaviour of giornoDellaSettimana
    - It may be the JUnit testing method we thought of in top-down and bottom-up strategies.
  - a stub acting as convert in the cases needed by the driver
    - It may be the stub we thought of in bottom-up testing



Limitations and problems of the technique with stubs and drivers

- Difficult application in object oriented
  - A stub can easily replace only a combinatorial method (without state)
    - If the method reads or modifies attributes of the class, additional stub (or real) methods must be implemented that perform these operations.
  - Each test case needs a different stub
    - Stubs replace methods, so before running each test we should rename the stub method (to make it replace the original method) and recompile
      - Poor support to test automation

Limitations and problems of the technique with stubs and drivers

- Drivers can also be made as JUnit test cases, but stubs seem to be annoying pieces of code that must be used/ignored when running each test.

- Which solutions can we achieve to avoid being forced to manage these explicit stubs?
  - Can we implement a test case entirely in a Junit test method code?

Test Doubles

Mock objects

Test Doubles

- A test double is an alternative and replacement implementation of an interface or class that could not be used for different possible reasons:
  - Too slow
  - Not (yet) available
  - Dependent on something else that is not (yet) available
  - Too difficult to instantiate and configure for testing
  - Involves unrecoverable operations
  - Interacts with some unpredictable resources (e.g. random generators or network connection)

- Only in the special case of a function, a double test of it can be a stub

Mock-Objects

- A precise emulation of the behavior of classes not yet implemented is obtainable with mock objects

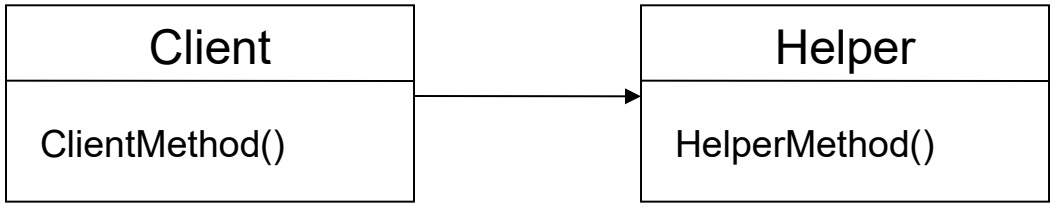
- A mock object is a simulation of a real object.
- It implements the interface of the object to be simulated and has its own behavior.
- It can provide fixed answers (such as a stub)
- It can check if the object that uses them does it correctly.

Dependency Injection and Testing

- Dependency Injection is fundamental for isolation and integration testing
  - It allows to perform a test on an object having entered as input a fictitious object on which it acts, replacing the concrete object that may not be available or not yet tested

Testing Classes with Mocks

E.g.: Client class (to be tested) uses Helper method



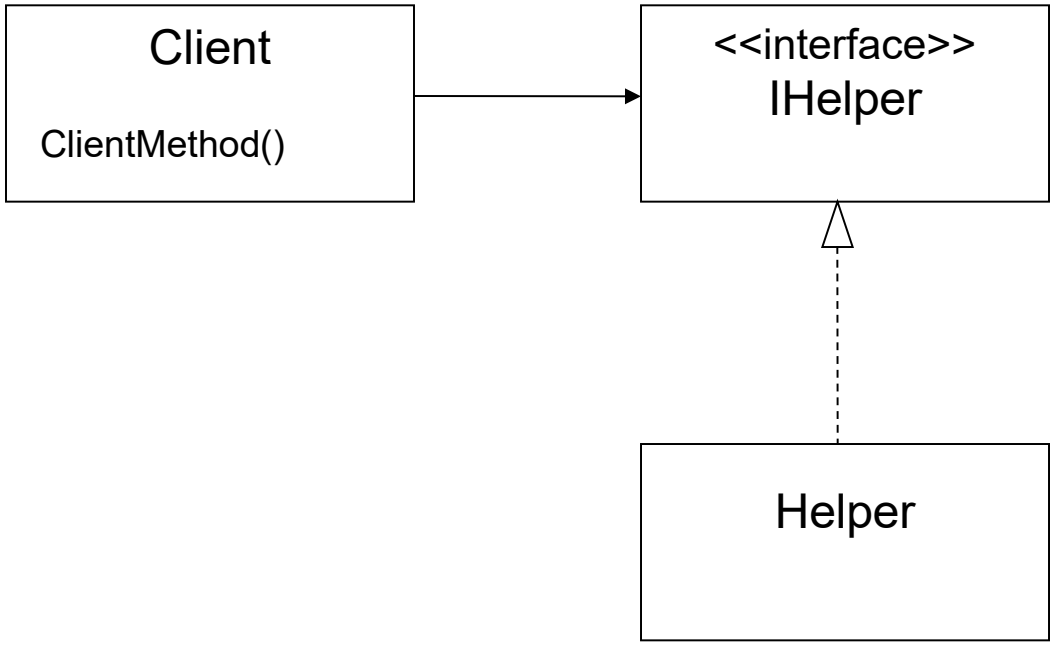
But the Helper class cannot be used because:

- It is not yet available
- We want to directly control what Helper returns to Client

- We want to build a Helper Test Double

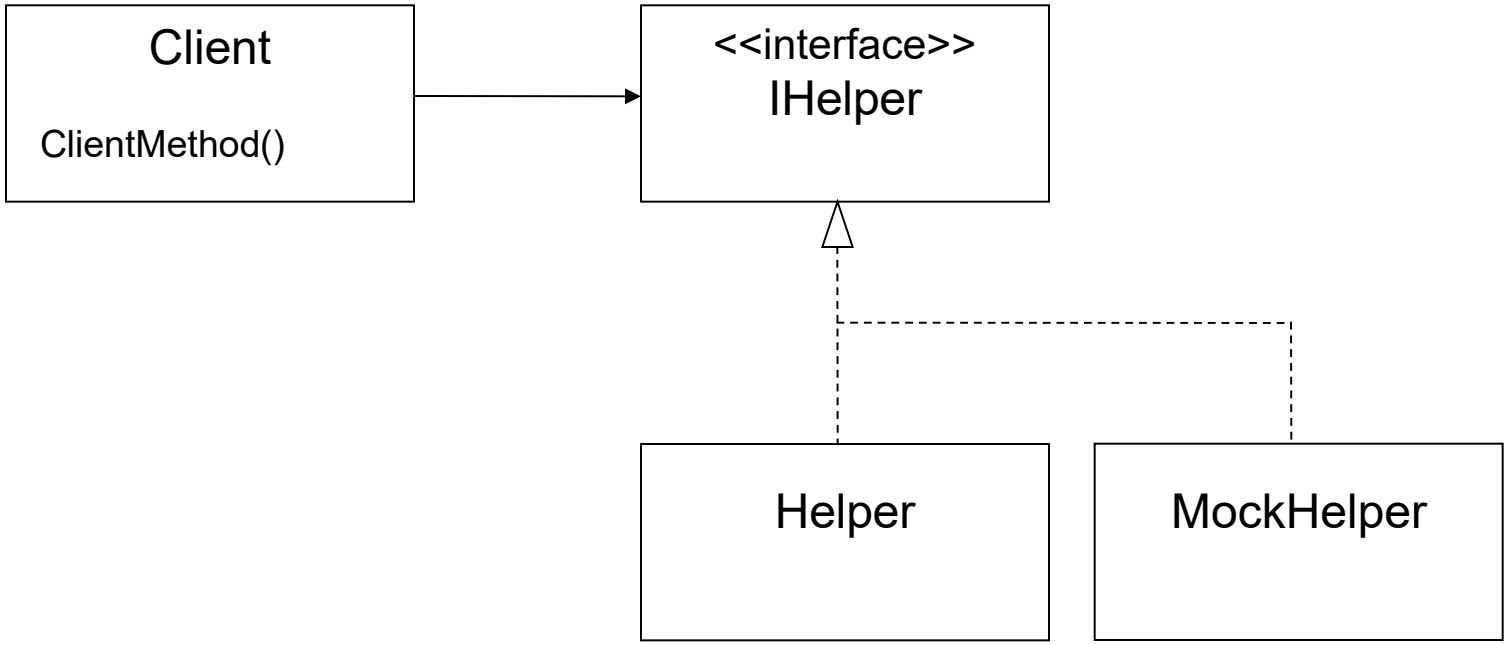
Mock-objects solution

We extract IHelper Interface



Mock-objects solution

The Mock-Object implements the IHelper interface



An example of a simple Mock

```
public class MockHelper implements IHelper
{
  public MockHelper ( )
  {
  }

  public Object helperMethod( Object aParameter )
  {
    Object result = null;
    if ( ! aParameter.toString().equals("expected" )
    {
      throw new IllegalArgumentException(
          "Unexpected parameter: " + aParameter.toString() );
    }
    result = new String("reply");
    return result;
  }
}
```

Using MockHelper

- A test class that tests the Client class using MockHelper instead of Helper will:
  - Instantiate a MockHelper object
  - Pass it to the Client constructor or to a Client setHelper method

- In this way, the original Helper class can be integrated in the future without the need to modify or destroy anything.

Examples of test methods

- In case the helper object can be passed through a constructor:

```
MockHelper m=new MockHelper();
Client c=new Client(m);
…
```

- In case the helper object can be set with a set method:

```
MockHelper m=new MockHelper();
Client c=new Client();
c.setHelper(m);
…
```

Example: Test Class

- To make this complex functional test case executable, it was enough to write PricingServiceTestDouble instead of PricingService
- **Note that the replacement between real object and its doubles takes place only during testing, not in the original PricingService**

```
public class OrderProcessorTest {
@Test
    public void testOrderProcessorWithMockObject() throws Exception {
        float initialBalance = 100.0f;
        float listPrice = 30.0f;
        float discount = 10.0f;
        float expectedBalance = (initialBalance - (listPrice * (1 - discount/100)));
        Customer customer = new Customer(initialBalance);
        Product product = new Product("TDD in Action", listPrice);
        OrderProcessor processor = new OrderProcessor();
        PricingService service = new PricingServiceTestDouble(discount);
        processor.setPricingService(service);
        processor.process(new Order(customer, product));
        assertEquals(expectedBalance, customer.getBalance(), 0.001f);
    }
}
```

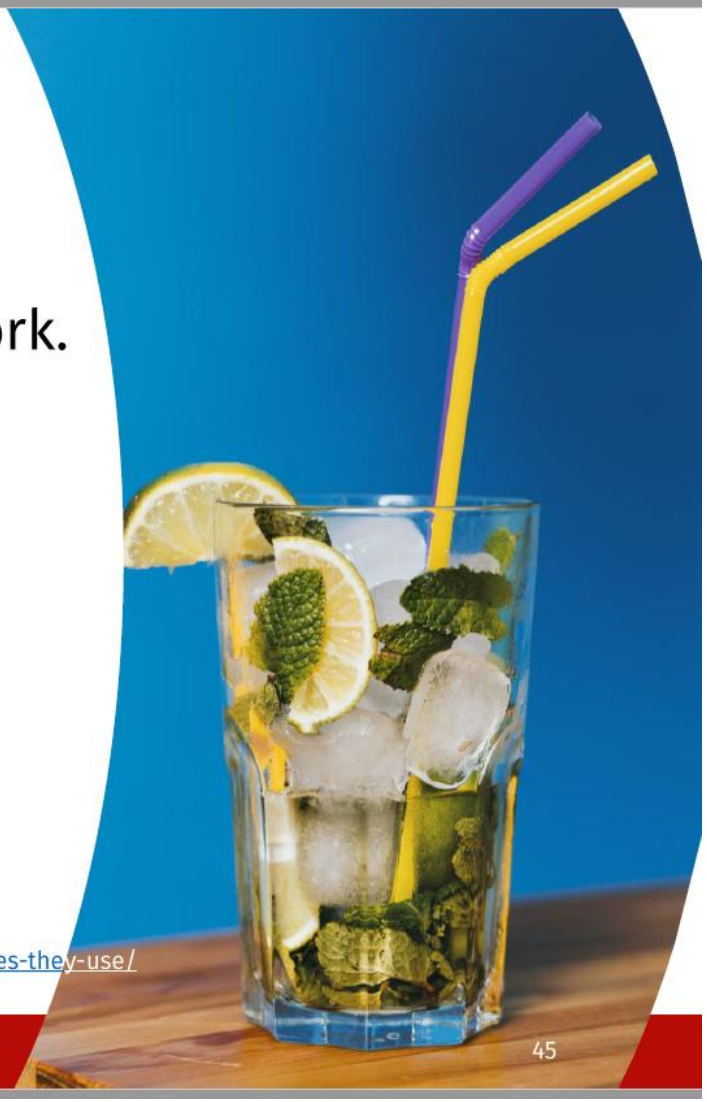
Problems with Mock Objects

- Mock Objects are quite bulky:
  - For each test case we should create different mock classes and, within the test case, instances of mock objects
- This solution would fill the code with a lot of classes and would be difficult to maintain
-
- We want a solution that creates mock classes anonymously and at runtime, without having to explicitly write all the code for the class

Introducing Mockito

Mockito is a well-known mocking framework.
Top 10 Java library across all libraries[1] .

Plus, it tastes really good!



[1] https://blog.overops.com/githubs-10000-most-popular-java-projects-here-are-the-top-libraries-they-use/

Mockito – Creating Mocks

Mocks can be created with the static method

`<T> T mock(Class<T> classToMock)`

By default, each method of the mock will return the default value for the corresponding type.

```
List mockedList = mock(ArrayList.class); // with classes
Map mockedMap   = mock(Map.class);       // and with interfaces too

mockedMap.get("Foo"); // returns null
mockedList.get(2);    // returns null
mockedList.isEmpty(); // returns false
mockedList.size();    // returns 0
```

Mockito – Configuring Mocks

We can configure the behaviour of our mock as follows:

```
List mockedList = mock(ArrayList.class); // with classes
Map mockedMap   = mock(Map.class);       // and with interfaces too

// configuration
when(mockedMap.get("Foo")).thenReturn("Bar");
doReturn("Bar").when(mockedList).get(2); // alternative way
when(mockedList.size()).thenReturn(99);

mockedMap.get("Foo"); // returns "Bar"
mockedList.get(2);    // returns "Bar"
mockedList.isEmpty(); // returns false as before
mockedList.size();    // returns 99
```

Mockito – Argument matchers

Often, we won't need to be so specific about input arguments.

In fact, being more generic could help us write less configuration code and more flexible tests.

```
// configuration: order matters!
when(mockedList.get(anyInt())).thenReturn("Mockito!");
when(mockedList.get(lt(0))).thenThrow(IllegalArgumentException.class);
when(mockedList.get(-25)).thenReturn("MOCKITO!");

mockedList.get(1988);  // returns "Mockito!"
mockedList.get(-25);   // returns "MOCKITO!"
mockedList.get(-2);    // throws IllegalArgumentException
```

Mockito – Configuring Mocks

When we invoke a method on mock with a given list of parameters, Mockito tries to find the latest configuration rule that «matches», and applies that.

If no configuration rule is found, it defaults to default values.

Therefore, configuration order matters, and more general rules should be declared before the more specific ones.

Mockito – Argument matchers

Mockito features a lot of built-in argument matchers, that can often also be combined in a Hamcrest-like fashion.

E.g.: `any()`, `anyString()`, `anyCollection()`, `leq()`, `isNull()`, …

We won't examine them in detail today, but if you're interested you can check out the ArgumentMatchers and the AdditionalMatchers classes from the docs.

It's also possible to use Hamcrest matchers as Mockito argument matchers with the MockitoHamcrest class!

Mockito – Arg. matchers with Hamcrest

```
import static org.mockito.Mockito.*;
import static org.mockito.hamcrest.MockitoHamcrest.argThat;
import static org.hamcrest.Matchers.*;
import static it.unina.computerscience.softeng.junitdemo.examples.IsPalindrome.*;

// configuration - argThat is a static method from the MockitoHamcrest class
when(mockedList.add(argThat(
        is(palindrome()) // Hamcrest Matcher
))).thenReturn(true);

mockedList.add("Abe");  // returns false
mockedList.add("Bob");  // returns true
mockedList.add("Otto"); // returns true
```

Mockito – Configuring Answers

When mocking with Mockito, you're not limited to pre-determined return values.

- With the generic interface Answer, you can customize the behaviour of your mocks by specifying an action to be executed to compute the desired output.

Mockito – Verifying behaviour

Usually, in the assert phase of our tests, we focus on checking that the final state of the Class Under Test is the one we expect.

This is testing by side-effects, and often it is not enough!

```
// assert
assertTrue(c.getAsset("EUR").get().getAmount() == 3000.0);
assertTrue(c.getAsset("BTC").get().getAmount() == 11.0);
// hopefully, a call was made to ExchangeAPI.exchange() with the right params!
```

Mockito – Verifying behaviour

After you're done with your mock, you can further inspect it to check if and how it's been used!

Out-of-the-box, with no configuration required, a Mockito mock keeps track of every method call it receives.

You can verify a mock's past behaviour with the `verify()` method and its variants.

Mockito – Verifying behaviour: examples

```
// check that myMock.size() was called exactly once
verify(myMock).size();
// check that myMock.get(19) was called exactly once
verify(myMock).get(19);
// check that myMock.get was called exactly three times with any int arg.
verify(myMock, times(3)).get(anyInt()); // with Mockito matcher
// check that myMock.get was called no more than three times with any even arg.
verify(myMock, atMost(3)).get(argThat(is(even()))); // with Hamcrest matcher
// check that myMock.get(0) was called at least two time
verify(myMock, atLeast(2)).get(0);
// check that no interaction occurred with mockedStack
verifyNoInteractions(mockedStack);
// check that myMock.addAll was never called
verify(myMock, times(0)).addAll(anyCollection());
```

Mockito – Verifying behaviour: examples

```
// check that method calls happened in a precise order
InOrder inOrder = inOrder(myMock);
inOrder.verify(myMock).get(19);
inOrder.verify(myMock).get(42);
inOrder.verify(myMock).get(99);

// check that no interaction happened after myMock.get(99)
verifyNoMoreInteractions(myMock);
```

Mockito – Verifying behaviour: errors

Here's what a `verify()` error message looks like:

```
> Error message:

org.mockito.exceptions.verification.NoInteractionsWanted:
No interactions wanted here:
-> at CryptocurrencyManagerTest.java:153
But found this interaction on mock 'arrayList':
-> CryptocurrencyManagerTest.java:138
```

Getting back at our example

```
@Test
void testTrading() {
    // create and configure mock
    ExchangePlatformAPI api = mock(ExchangePlatformAPI.class);
    when(api.exchange("EUR", 7000.0, "BTC")).thenReturn(new Asset("BTC",1.0));
    // arrange
    CryptocurrencyManager c = new CryptocurrencyManager(api);
    c.addAsset(new Asset("BTC",10.0)); c.addAsset(new Asset("EUR",10000.0));
    // act
    c.trade("EUR", 7000.0, "BTC");
    //assert
    assertTrue(c.getAsset("EUR").get().getAmount() == 3000.0);
    assertTrue(c.getAsset("BTC").get().getAmount() == 11.0);
    verify(api).exchange("EUR",7000.0,"BTC"); // API were actually called
}
```

Advanced support for assertions

Hamcrest

http://hamcrest.org/JavaHamcrest/

Hamcrest

- **Hamcrest is a framework (library) supporting the efficient implementation of assertion**
- **It completes the Assert classes of JUnit allowing the realization of assertions that are:**
  - More powerful
  - Easier to read
- **Hamcrest assertion can be useful also outside the scope of unit testing**
- **http://hamcrest.org/JavaHamcrest/**

assertThat

- **The entry point of Hamcrest is the assertThat method that can be used in substitution to all the assert ones of JUnit**

```
public static <T> void assertThat
(T actual, Matcher<? super T> matcher)
```

- **Where T can be any type or classes of Java and Matcher is a comparison function applied to the expected value**

Examples

```
assertEquals (expected, observed); →
  assertThat (observed, equalTo(expected);
```

```
assertTrue (condition); →
  assertThat (condition, is(true));
```

```
assertNotNull(observed); →
  assertThat (observed, is(not(nullValue())));
```

Matchers

- **equalTo (o) : perform an object comparison (according to the defined o.equalTo method)**
- **not (m) : it returns the complement of Matcher m**
- **is (m) : similar to equalTo but can be applied also on a Matcher (in this case it does nothing)**
  - for readability purposes :
    - *assertThat (" foo", is(not(equalTo (observed))))*

String Matchers

- **containsString(s)**
- **containsStringIgnoringCase(s)**
- **startsWith(s)**
- **endsWith(s)**
  - And corresponding versions with IgnoringCase

Comparators Matchers

- **greaterThan(c)**
- **greaterThanOrEqualTo(c)**
- **lessThan(c)**
- **closeTo(c,delta)**
  - For comparisons involving float, double, etc.

Logical Matchers

- **allOf(m1, ...)**
- **anyOf(m1, ...)**
  - Taking as parameters a list of matchers
- **both(m1).and(m2)**
- **either(m1).or(m2)**

Object attribute matchers

- **hasProperty(s)**
  - Satisfied if the object has an attribute named s
    - *s is the name of the attribute, not its value!*
- **hasProperty(s,m)**
  - Satisfied if the object has an attribute named s and this attribute satisfies the matcher m

```
Ticket t = new Ticket ( “Napoli-Ajax”, 70, “Distinti"); //match, cost, sector
assertThat (t , hasProperty(“sector”);
assertThat (t , hasProperty (“cost", is(greaterThan(50))));
```

Collection Matchers

- **hasItem (m)**
  - Repeats the matcher evaluation on each item of the collection and is satisfied if it exists at least one matching item
- **contains (m1, m2, ...)**
  - These matcher has a list of matchers as parameter: it is satisfied if the first item of the collection satisfy m1, the second one satisfy m2, an so on
- **containsInAnyorder (m1, m2, ...)**
  - Similar to the previous one, but it is satisfied if there exist exactly one item satisfying m1, one satisfying m2 and so on

Collection Matchers Examples

```
List<String> s = Arrays.asList("Uno, Due, Tre");

assertThat (s , hasItem(both(startsWith("D")).and(endsWith("e"))));

assertThat (s , contains( startsWith ("U"),startsWith("D"), startsWith("T")));

assertThat (s , containsInAnyOrder("Tre","Uno","Due"));
```

Maps matchers

- **hasKey(m)**
  - Checks the name of each key of the map item
- **hasValue (m)**
  - Checks the value of each map item
- **hasEntry(m1, m2)**
  - Checks the matcher m1 on each key and the matcher m2 on the corresponding item value

Customized Matchers

- **New matchers can be created by implementing the matchers interface**

```
public interface Matcher<T> {
   boolean matches(Object actual);
   void describeMismatch(Object actual, Description
   mismatchDescription);
}
```

- **The describeMismatch method is very useful to improve the quality of the returned error, giving more information for the debugger**
- **It is now more convenient to extending more specialized abstract classes, such as TypeSafeMatcher<T>**

Customized Matcher Example

```
public class IsPalindrome extends TypeSafeMatcher<String>{
    @Override
    public void describeTo(Description description) {
        description.appendText("a palindrome string");
    }
    @Override
    protected Boolean matchesSafely(String item) {
       StringBuilder sb= new StringBuilder(item).reverse();
       return sb.toString().equalsIgnoreCase(item);
    }
    public static IsPalindrome palindrome() {
       return new IsPalindrome();
    }
}
…
assertThat(s, is(palindrome()));
```